# 1. Hyrje në algoritme

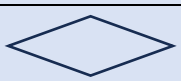
**- Algoritmet** janë hapat e rregullt dhe të qartë që përdoren për të zgjidhur një problem ose për të përpunuar të dhëna. Ato janë baza e çdo programi kompjuterik dhe lejojnë që një problem kompleks të ndahet në pjesë të thjeshta, të zbatueshme dhe të verifikueshme.

**- Studimi i algoritmeve** është i rëndësishëm sepse ato përdoren në fusha të ndryshme: internet (motorë kërkimi, rrjete sociale), biologji, kompjuterikë (kompajlerë, sisteme skedari), grafikë kompjuterike (lojëra, filma, realitet virtual), siguri (kriptografi, pagesa online) dhe shumë të tjera. Algoritmet gjithashtu ndihmojnë për zhvillimin e aftësive intelektuale dhe logjike, duke stimuluar mënyrën e të menduarit të saktë dhe të organizuar.

**- Analiza e algoritmeve** ka për qëllim të vlerësojë sa efikas dhe i saktë është një algoritëm, sipas:
**Korrektësisë:** një algoritëm quhet i saktë nëse jep gjithmonë rezultatin e duhur për çdo input të mundshëm. Korrektësia provohet zakonisht duke përdorur rastet e testimit, provime matematikore ose argumente logjike që tregojnë se çdo hap i algoritmit është i drejtë.
**Efikasitetit** që matet kryesisht në dy mënyra:

- **Efikasiteti kohor (**Time Complexity**):** tregon sa kohë i duhet algoritmit për të përfunduar në varësi të madhësisë së inputit *n*. Shpesh shprehet me notacionin **Big O**, p.sh.$O(n)$, $O(n^2)$, $O(\log_n)$.
- **Efikasiteti hapësinor (**Space Complexity**):** tregon sa memorie i duhet algoritmit për të përpunuar inputin dhe për të ruajtur të dhënat e ndërmjetme. Përfshin variablat, matricat, listat, stekat, etj.

**Flowchart:** një mënyrë vizuale për të treguar hapat e algoritmit. Përdoren simbolet standarde: drejtkëndëshat për veprime, rombet për vendimmarrje, dhe shigjetat për rrjedhën.

| Simboli | Përdorimi |
|---|---|
|  | Fillimi i një algoritmi |
|  | Futja apo leximi i variablave |
|  | Llogaritje të vlerave |
|  | Shtypja e vlerave të futura |
|  | Pyetje Po/Jo që përcakton degëzimin e veprimeve të mëtejme |
|  | Fundi i një algoritmi |

- **Pseudocode (**tekstual**)** është një version i afërt me kodin, por pa sintaksë strikte të një gjuhe programimi. Ajo lehtëson analizimin e logjikës së algoritmit.

**- Kodi real** është mplementimi në një gjuhë programimi (Java, Python, C++, etj), i cili përdoret për testim praktik dhe matje të kohës së ekzekutimit.

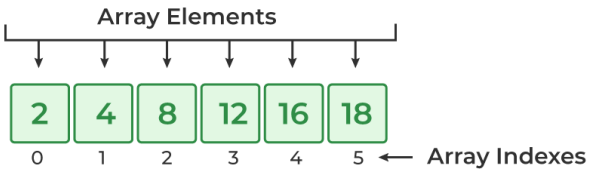
# 2. Tipet e të dhënave

- Algoritmet përdoren për **kërkim (***search***)** dhe **renditje (***sortim***)** të të dhënave. **Renditja** (sorting) përdoret për të vendosur të dhënat në një rend logjik (p.sh. nga më e vogla te më e madhja), ndërsa **kërkimi** (searching) bëhet më i shpejtë pasi të dhënat janë të renditura.
Pra, fillimisht bëjmë sortim dhe pastaj kërkim, në mënyrë që algoritmet të jenë më efikase. Për të zgjidhur probleme të ndryshme, përdorim kombinime të algoritmeve të sortimit dhe kërkimit, të mbështetura në struktura të të dhënave të përshtatshme.

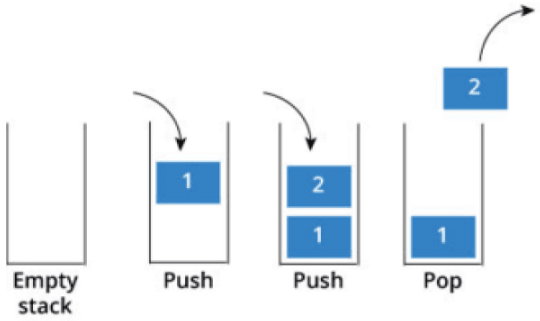
- Programet ekzekutohen në **CPU**, por për t'u ekzekutuar ato kanë nevojë për të dhëna, të cilat vijnë nga RAM-i. Të dhënat në RAM organizohen përmes strukturave të të dhënave, të cilat përcaktojnë mënyrën se si ruhen, qasen dhe modifikohen informacionet.

**- Strukturat e të dhënave (***lista, array***)** janë mënyra të organizimit të informacionit në memorje. Ato njihen edhe si **koleksione të objekteve** (*collections of objects*) dhe përfshijnë tipe si: **array**, **stack** dhe **queue**.
Këto struktura njihen si **ADT (***Abstract Data Types***)** janë koncepte logjike që përfaqësojnë mënyrën se si ruhen dhe manipulohen të dhënat, por nuk ekzistojnë fizikisht në memorien RAM.

**1. Array** është një koleksion elementesh të tipit të njëjtë dhe ka madhësi statike që do të thotë se madhësia e tyre përcaktohet në momentin e krijimit dhe nuk mund të ndryshohet më pas. . Qasja në elementet e array-t bëhet përmes **pozitës (***indeksit***)** i cili është një numër i plotë (integer). Kjo qasje është primitive dhe shumë e shpejtë, sepse çdo element mund të arrihet direkt përmes indeksit. Zakonisht përdorim loop për të iteruar mbi elementet e një array.




**2. Stack (LIFO –** *Last In First Out***)** është një strukturë e të dhënave ku elementi i fundit që futet është i pari që del. Mund ta imagjinojmë si një grumbull pjatash të lara, e para që do hiqet është ajo që është vendosur e fundit. Operacionet bazë të stack-ut:




- **push(item)** – shton një element në krye të stack-ut
- **pop()** – heq dhe kthen elementin më të fundit
- **isEmpty()** – kontrollon nëse stack-u është bosh
- **size()** – tregon numrin e elementeve në stack

**Implementimi** i stack mund të bëhet në dy mënyra:

**- Linked List** çdo element (**node**) lidhet me tjetrin përmes një pointeri. Ajo mund të krahasohet me një tren, ku çdo vagon (**node**) është një element që përmban të dhëna dhe një **referencë** (next), e cila tregon drejt elementit tjetër në listë. Është me madhësi dinamike, mund të shtojmë ose të heqim elemente sa herë të duam pa pasur nevojë për një madhësi fikse.

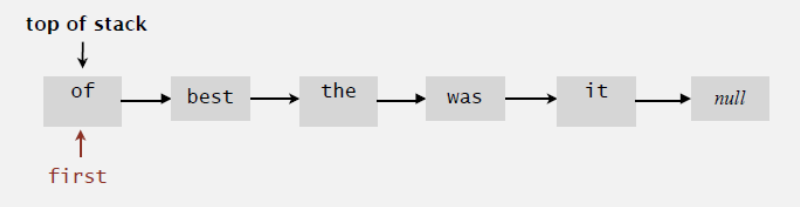
- Një node është një objekt që ruan të dhëna dhe referencën për node-in pasardhës.
- Nëse një node nuk ka referencë (pra është null), nuk mund t'i qasemi më, kjo do të thotë se ai element është "humbur" në memorien RAM.
- Lidhja ndërmjet node-ve realizohet përmes variablës së rezervuar next.

Ekzistojnë disa lloje të Linked List:

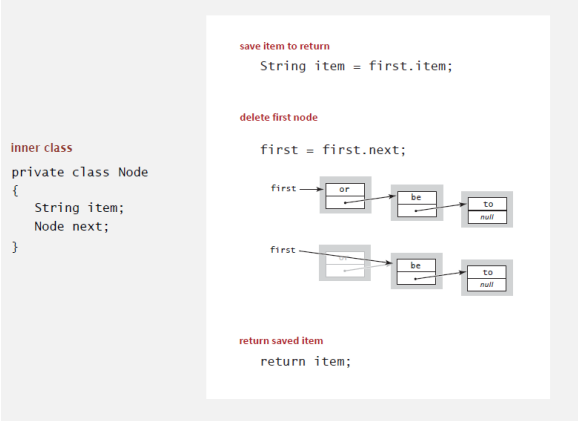
- **Singly Linked List** – çdo node ka vetëm një referencë (next).
- **Doubly Linked List** – çdo node ka dy referenca: një për node-in next dhe një previous.
- **Circular Linked List** – node-i i fundit lidhet me të parin, duke krijuar një cikël.

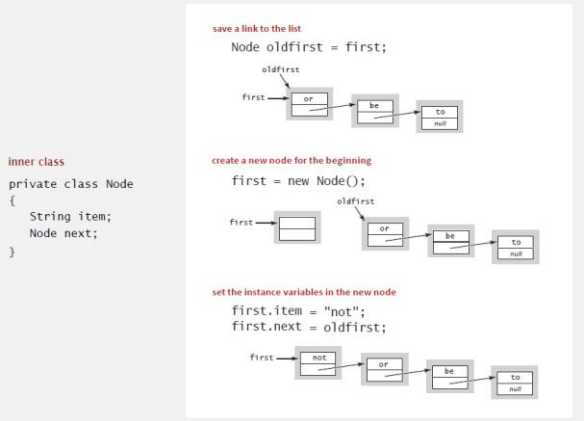
Kur Stack implementohet përmes një Linked List, elementi më i rëndësishëm është top of the stack, sepse aty ndodhen të dhënat me të cilat punojmë (për shembull, në operacionet push dhe pop).

- **Push** shton një element të ri në majë të stack-ut.
- **Pop** heq elementin e fundit të futur, pra atë që ndodhet në majë. Në këtë rast, node i fundit në stack shpesh quhet first, sepse është ai që qasemi i pari.

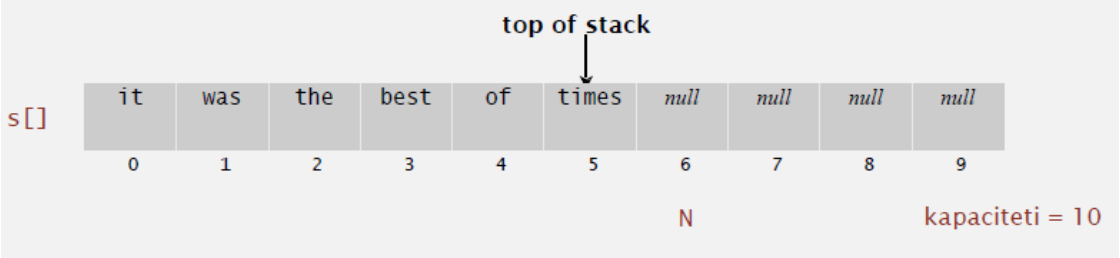



**Fshirja** në Linked List bëhet duke hequr referencat që lidhin node-t. Nëse kemi tre node të lidhura dhe heqim referencën e parë, nuk mund t'i qasemi më node-it të dytë, ky fenomen quhet **loitering**, që do të thotë se një pjesë e memories është e zënë, por e paarritshme. Kur heqim referencën që lidh node-in e parë me të dytin, node-i i parë mund të lidhet drejtpërdrejt me node-in e tretë.



**- Array me kapacitet të fiksuar** rritet dinamikisht me "resizing array" kur është plot. Një array me kapacitet 10 që përmban 6 elemente. Nëse ndodh overflow (kur tejkalohet kapaciteti), atëherë zakonisht rritet dyfish madhësia e array-t. Nëse ndodh underflow (kur kemi shumë pak elemente), madhësia mund të zvogëlohet për ¼ për të optimizuar hapësirën.




Përdoret array s[] për të ruajtur N items në stack.

- **push()** – shtoi tem të ri në s[N].
- **pop()** – me rritem prej s[N-1].

**- Rreziqet** gjatë ekzekutimit të programit me strukturën stack**:**

- **Overflow** – kur mundohemi të shtojmë element kur stack është plot.
- **Underflow** – kur mundohemi të heqim nga një stack bosh.

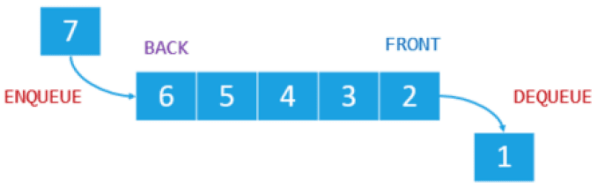
Për këtë arsye përdoret isEmpty() përpara se të bëjmë pop().

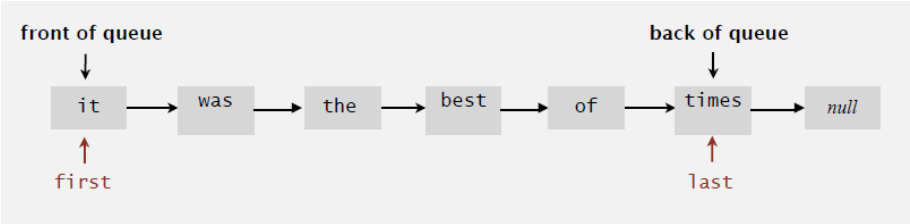
**Shembuj** të përdorimit të Stack-ut:

- Undo/Redo në dokumente
- Butoni "Back" në browser
- Thirrje funksionesh në compiler
- Llogaritje aritmetike (p.sh. algoritmi i Dijkstra-s)

**3. Queue (FIFO –** *First In First Out***)** funksionon si një **rresht**, elementi i parë që futet është i pari që del. Kjo përdoret kur rendi i futjes së elementeve duhet të ruhet. **Operacionet** kryesore:

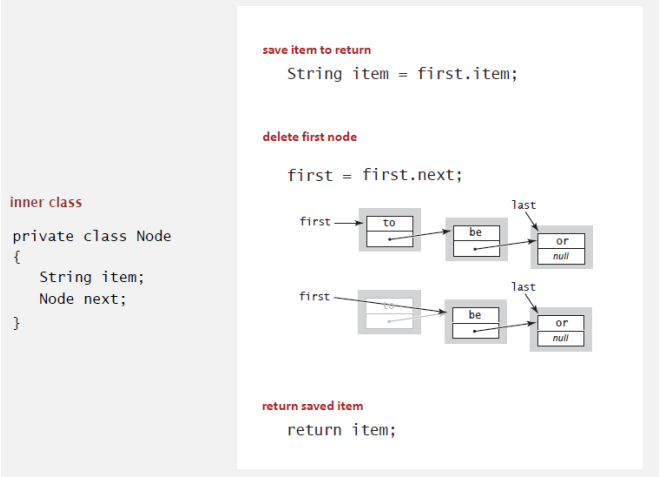
- **enqueue(item)** – shton një element në fund të queue
- **dequeue()** – heq elementin e parë
- **isEmpty()** – kontrollon nëse është bosh
- **size()** – numëron elementet

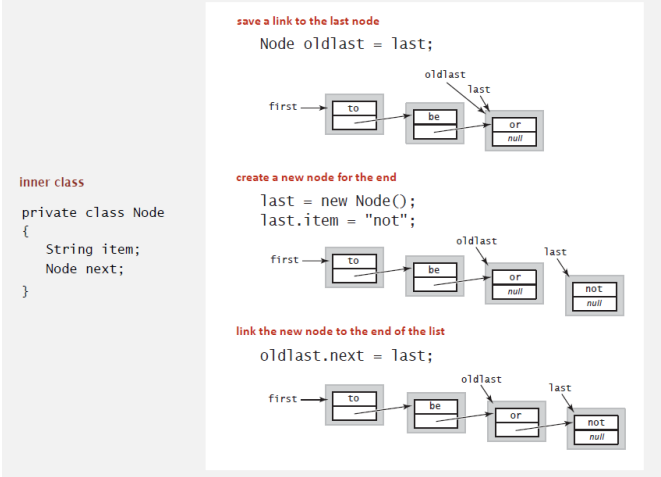


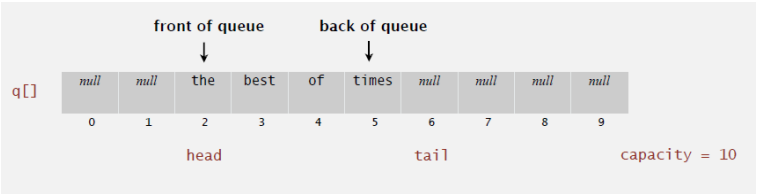
**Implementimet** e Queue mund të bëhet në dy mënyra:

- **Linked List:** ka pointer tek elementi i parë (first) dhe i fundit (last). Komanda **enqueue()** shton në fund, kurse **dequeue()** heq nga fillimi, dhe të dy operacionet kanë **kohë konstante (**O(1)**)**.




- **Array Resizable:** elementet ruhen në një array ku përdoren dy indekse: **head** dhe **tail**. Kur array mbushet, dyfishohet automatikisht. Kur është vetëm 25% e mbushur, përgjysmohet për efikasitet.




**-** Çdo Stack dhe Queue janë **strukturë të dhënash abstrakte (***ADT***)** që mbështesin këto operacione:

- **Insert** – fut të dhëna
- **Remove** – heq të dhëna
- **Iterate** – shfleton ose kërkon të dhëna
- **Test if empty** – kontrollon nëse është bosh për të shmangur gabime (crash/error) gjatë heqjes së elementeve.

**- API (Application Programming Interface)** është një **ndërfaqe programimi** që lejon komunikimin midis dy programeve. Në kontekstin e strukturave të të dhënave, një API përcakton metodat dhe funksionalitetet që mund të përdorim (si push, pop, enqueue, dequeue), pa pasur nevojë të dimë implementimin e brendshëm.

**-** Në Java përdoren **Generics** për të krijuar struktura të sigurta, p.sh. Stack<Apple> që pranon vetëm objekte të tipit Apple. Kjo ndihmon në shmangien e gabimeve gjatë ekzekutimit (run-time errors). Për më tepër, implementimet moderne përdorin Iterable dhe Iterator, që lejojnë:

- **hasNext()** për të kontrolluar nëse ka më shumë elemente, dhe
- **next()** për të kaluar tek tjetri.

Kjo e bën më të lehtë iterimin mbi një strukturë të dhënash me foreach.

**- Performanca dhe memoria**

- Implementimi me **linked list** përdor rreth 40 bytes për çdo node (*pa llogaritur objektet që referohen*).
- Operacionet e **push()** dhe **pop()** kanë kohë konstante (O(1)) në rastin më të keq.
- Në **array dinamik**, madhësia e array dyfishohet kur është plot dhe përgjysmohet kur mbetet 25% e mbushur, për të ruajtur balancën midis hapësirës dhe efikasitetit.

**- Përmbledhje e Konceptit**

- **Array** – qasje direkte përmes indeksit.
- **Stack** – LIFO, përdoret kur duam të rikthehemi pas ose të ruajmë rend të kundërt.
- **Queue** – FIFO, përdoret kur duam të ruajmë rendin e ardhjes.
- Të gjitha janë struktura të të dhënave abstrakte (ADT) që përkufizohen përmes API-ve dhe implementohen në mënyra të ndryshme.

## 3. Analiza e perfomancës së algoritmeve

### - Parashikimi dhe Analiza e Performancës së Algoritmeve

Analiza e algoritmeve është procesi që ndihmon programuesit të kuptojnë se sa efikas dhe korrekt është një program. Dy aspektet kryesore të performancës janë **korrektësia** dhe **efikasiteti**. Një algoritëm është korrekt nëse jep gjithmonë rezultatin e duhur për çdo input të vlefshëm, ndërsa është efikas nëse e arrin këtë rezultat me kohë dhe burime minimale.

### - Arsyet për të Analizuar Algoritmet

Analiza e algoritmeve është e domosdoshme për të krahasuar performancën midis zgjidhjeve të ndryshme dhe për të shmangur "performance bugs", raste kur programi është teknikisht i saktë, por shumë i ngadaltë për t'u përdorur në praktikë. Një arsye tjetër është parashikimi i sjelljes së programit me input të madh, siç pyeste Knuth: *"A do të jetë programi në gjendje të zgjidhë probleme me të dhëna shumë të mëdha?"* Kjo kërkon përdorimin e metodave shkencore në analizë.

Në analizën e performancës përdoren metoda të ngjashme me ato të shkencave natyrore:

- **Vëzhgimi** – identifikimi i një fenomeni ose sjelljeje në program.
- **Hipoteza** – ndërtimi i një modeli që shpjegon vëzhgimin.
- **Parashikimi** – përdorimi i modelit për të parashikuar performancën.
- **Verifikimi** – testimi i modelit përmes eksperimenteve të reja.
- **Validimi** – përsëritja e procesit derisa rezultatet të përputhen me parashikimet.

Eksperimentet duhet të jenë të riprodhueshme, dhe çdo hipotezë duhet të jetë e mohueshme, pra të mund të hidhet poshtë nëse nuk përputhet me rezultatet.

### - Modeli Matematikor i Kohës së Ekzekutimit (Running Time)

Koha totale e ekzekutimit përbëhet nga shuma e çmimit të operacioneve shumëzuar me frekuencën e ekzekutimit.

- **Çmimi** i një operacioni varet nga makina (hardware-i) dhe kompajlleri.
- **Frekuenca** varet nga algoritmi dhe nga madhësia e inputit.

Sipas Donald Knuth, është e mundur të krijohen modele matematikore të sakta që përshkruajnë kohën e ekzekutimit për algoritmet.

### - Çmimi i Operacioneve Themelore

Operacionet bazë si shtimi, shumëzimi, ndarja, ose krahasimi zakonisht kanë **kohë konstante** të ekzekutimit. Për shembull, në një MacBook Pro 2.2GHz:

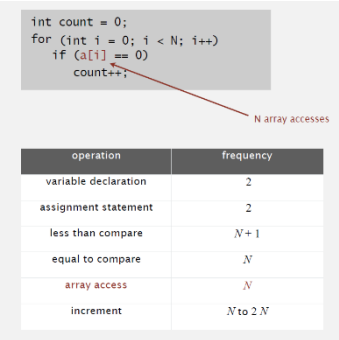
| Operation | Example | Nanoseconds† |
|---|---|---|
| integer add | a+b | 2.1 |
| integer multiply | a*b | 2.4 |
| integer divide | a/b | 5.4 |
| floating-point add | a+b | 4.6 |
| floating-point multiply | a*b | 4.2 |
| floating-point divide | a/b | 13.5 |
| sine | Math.sin(theta) | 91.3 |
| arctangent | Math.atan2(y,x) | 129.0 |

Operacionet si krijimi i një array-i **njëdimensional** (new int[N]) kanë kosto që varet nga N, ndërsa për një **array dy-dimensional**, kostoja është afërsisht $N^2$.

Shumica e operacioneve primitive marrin kohë konstante.

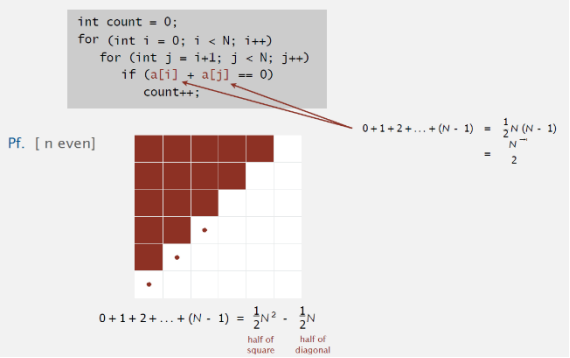
| Operation | Example | Nanoseconds† |
|---|---|---|
| variable declaration | inta | $c1$ |
| assignment statement | a=b | $c2$ |
| integer compare | a<b | $c3$ |
| array element access | a[i] | $c4$ |
| array length | a.length | $c5$ |
| 1D array allocation | newint[N] | $c_6\ N$ |
| 2D array allocation | newint[N][N] | $c_7\ N^2$ |

**- Shembuj të Llogaritjes së Instruksioneve**




**1-SUM:** Ky funksion numëron sa elemente në një array janë zero:
Numri i instruksioneve rritet në mënyrë **lineare** me N.

**2-SUM:** Ky funksion kontrollon çdo çift elementesh për të parë nëse shuma e tyre është zero:
Kjo kërkon **rreth N² hapa** (kompleksitet kuadratik).




| operation | frequency |
|---|---|
| variable declaration | $N + 2$ |
| assignment statement | $N + 2$ |
| less than compare | $\frac{1}{2}(N + 1)(N + 2)$ |
| equal to compare | $\frac{1}{2} N (N - 1)$ |
| array access | $N (N - 1)$ |
| increment | $\frac{1}{2} N (N - 1)$ to $N (N - 1)$ |

```
int count = 0;
for (int i = 0; i < N; i++)
   for (int j = i+1; j < N; j++)
      for (int k = j+1; k < N; k++)
         if (a[i] + a[j] + a[k] == 0)
            count++;
```

"inner loop"

$$\frac{N^3}{3} = \frac{N(N-1)(N-2)}{3!} \rightarrow \frac{1}{6}N^3$$

A. $\sim \frac{1}{2} N^3$ array qasje.

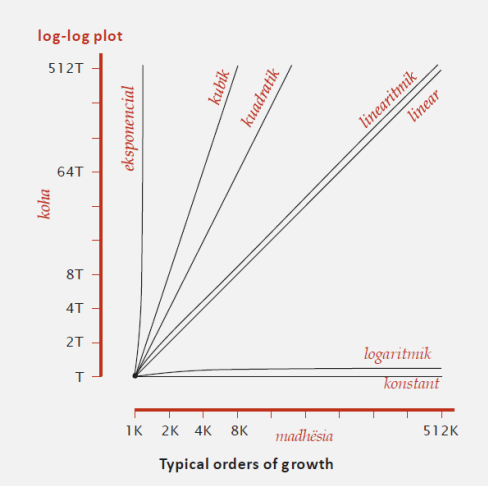
**3-SUM:** Nëse shtohet një cikël tjetër për të parë çdo treshe elementesh:

Tani kemi **rreth $N^3$ hapa**, që tregon një rritje shumë më të madhe të kompleksitetit kohor.

**- Shkalla e Rritjes (*Order of Growth*)**

Në analizë përdorim funksione si:

- **1** → konstant
- **log N** → logaritmik
- **N** → linear
- **N log N** → linearithmic
- **$N^2$** → kuadratik
- **$N^3$** → kubik
- **$2^N$** → eksponencial




Sa më e ulët të jetë shkalla e rritjes, aq më i **shpejtë dhe efikas** është algoritmi. Shembull: një algoritëm logaritmik është shumë më i shpejtë se një eksponencial kur N bëhet i madh.

| Order Of Growth | Name | Typical Code Framework | Description | Example | *T*(2*n*) /T(*N*) |
|---|---|---|---|---|---|
| 1 | **constant** | a=b+c; | statement | add two numbers | 1 |
| log *N* | **logarithmic** | while(N>1)<br>{N=N/2;...} | divide in half | binary search | ~ 1 |
| *N* | **linear** | for(inti=0;i<N;i++)<br>{...} | loop | find the maximum | 2 |
| *N* log *N* | **linearithmic** | [see mergesort lecture] | divide and conquer | mergesort | ~ 2 |
| *N* 2 | **quadratic** | for(inti=0;i<N;i++)<br>for(intj=0;j<N;j++)<br>{...} | double loop | check all pairs | 4 |
| *N* 3 | **cubic** | for(inti=0;i<N;i++)<br>for(intj=0;j<N;j++)<br>for(intk=0;k<N;k++)<br>{...} | triple loop | check all triples | 8 |
| 2*N* | **exponential** | [see combinatorial search lecture] | exhaustive search | check all subsets | *T*(*N*) |

**- Notacionet Asimptotike**

Këto notacione përdoren për të klasifikuar kompleksitetin:

- **Big O (O(f(N)))** → kufiri i sipërm (tregon rastin më të keq).
- **Big Ω (Omega)** → kufiri i poshtëm (rasti më i mirë).
- **Big Θ (Theta)** → kufiri i saktë asimptotik (rasti tipik/mesatar).

**Shembull i Krahasimit të Performancës**

Në testin e programeve ThreeSum.java dhe ThreeSumDeluxe.java, u vu re se rritja e N ndikon ndjeshëm në kohën e ekzekutimit:

| N | time (seconds) |
|---|---|
| 1,000 | 0.1 |
| 2,000 | 0.8 |
| 4,000 | 6.4 |
| 8,000 | 51.1 |

ThreeSum.java

| N | time (seconds) |
|---|---|
| 1,000 | 0.14 |
| 2,000 | 0.18 |
| 4,000 | 0.34 |
| 8,000 | 0.96 |
| 16,000 | 3.67 |
| 32,000 | 14.88 |
| 64,000 | 59.16 |

ThreeSumDeluxe.java

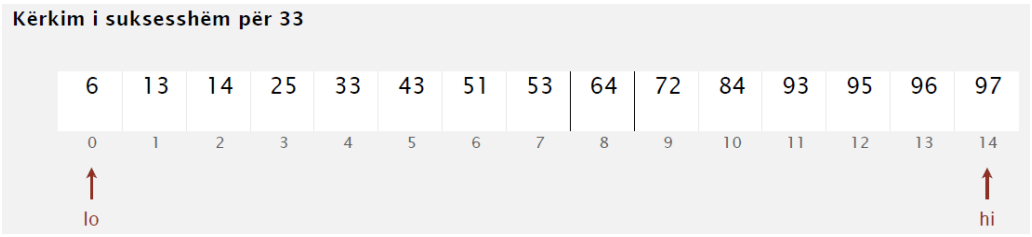
Ky eksperiment tregon se **sa shpejt rritet koha** kur rritet madhësia e inputit për algoritme me kompleksitet më të lartë. Hipotezë. Algoritmi sortues $N2 \log N$ për 3-SUM është dukshëm më I shpejtë realisht sesa brute-force $N3$ algoritmi.

Sa më e ulët order of growth, aq më i shpejtë programi

**- Binary Search** është një **algoritëm shumë efikas për kërkim** në një listë të renditur. Ai funksionon duke ndarë listën në gjysma:

- Krahasohet vlera që kërkohet me elementin e mesëm.
- Nëse është më e vogël → kërko majtas.
- Nëse është më e madhe → kërko djathtas.
- Nëse janë të barabarta → elementi është gjetur.

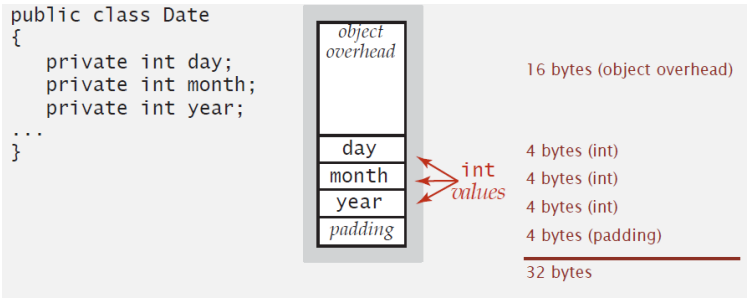
Kjo metodë ka kompleksitet **log N**, që e bën shumë më të shpejtë se kërkimi linear (N).



## Përdorimi i Memories në Java

### Tipe primitive

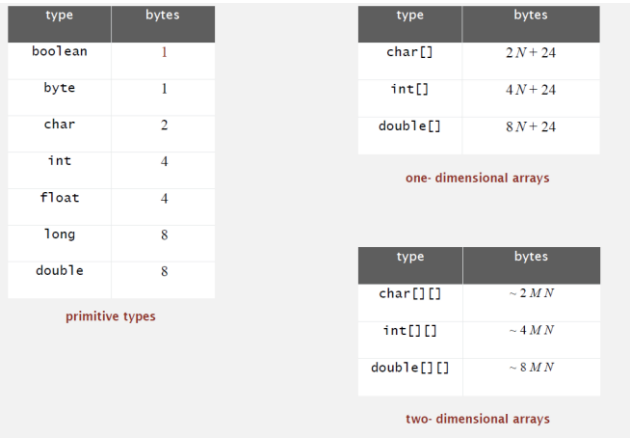
- int → 4 bytes
- double → 8 bytes
- boolean dhe byte → 1 byte
- char → 2 bytes




### Objektet

- Çdo objekt ka **16 bytes overhead**.
- Çdo **referencë** (pointer) e objektit zë 8 bytes.
- Përdoret edhe **padding**, që e rrumbullakëson madhësinë në shumëfish të 8 bytes. Shembull: një objekt Date me tre variabla int përdor rreth **32 bytes memorie**.

**Array-t** kërkojnë **24 bytes shtesë** për strukturën e tyre. Një array dy-dimensional int[N][N] përdor afërsisht $N^2$ elementë × 4 byte secili.



| type | bytes |
|---|---|
| boolean | 1 |
| byte | 1 |
| char | 2 |
| int | 4 |
| float | 4 |
| long | 8 |
| double | 8 |

primitive types

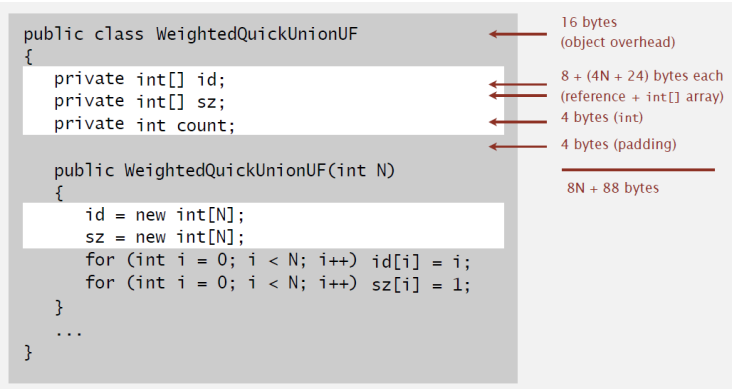
| type | bytes |
|---|---|
| char[] | $2N + 24$ |
| int[] | $4N + 24$ |
| double[] | $8N + 24$ |

one- dimensional arrays

| type | bytes |
|---|---|
| char[][] | $\sim 2MN$ |
| int[][] | $\sim 4MN$ |
| double[][] | $\sim 8MN$ |

two- dimensional arrays

### Shembull i Llogaritjes së Memories




Totali i memories është afërsisht **8N + 88 bytes**. Kjo përfshin overhead-in e objektit, referencat, array-t dhe padding-un.

# 5. Elementary sort

- Sortimi është procesi i **rirregullimit të një array-i me N elemente** sipas një renditje të caktuar, zakonisht nga më i vogli tek më i madhi. Shembuj:

- renditja e të dhënave të studentëve,
- listat e kontakteve.

Gjërat e sortuara gjenden më lehtë, kur elementët janë në rend, kërkimi, filtrimi dhe analizimi bëhen shumë më të shpejta. 3 sortimet elementare janë: **selection** sort, **insertion** sort dhe **shell** sort.

**- Comparable API dhe krahasimi i objekteve:** për të sortuar, duhet të dimë si **krahason** dy objekte të tipit të njëjtë. Implementohet compareTo() ashtu që **v.compareTo(w).** Në Java, kjo bëhet me metodat kryesore:

- **compareTo()** – kthen:
  - **0** nëse objektet janë të barabarta,
  - **-1** nëse objekti "v" është më i vogël se "w",
  - **+1** nëse "v" është më i madh se "w".
- **equals()** – kthen vetëm *true / false*.




Objektet duhet të implementojnë **Comparable**, që tregon çfarë çelësi po sortohet (p.sh. emri, mosha, nota).

```
public class Date implements Comparable<Date>
{
   private final int month, day, year;

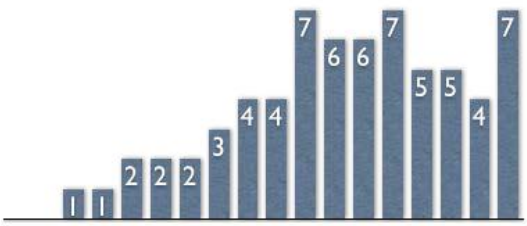
   public Date(int m, int d, int y)
   {
      month = m;
      day   = d;
   }  year  = y;


   public int compareTo(Date that)
   {
      if (this.year  < that.year ) return -1;
      if (this.year  > that.year ) return +1;
      if (this.month < that.month) return -1;
      if (this.month > that.month) return +1;
      if (this.day   < that.day  ) return -1;
      if (this.day   > that.day  ) return +1;
      return 0;
   }
}
```

Krahason vetëm dates me dates të tjera

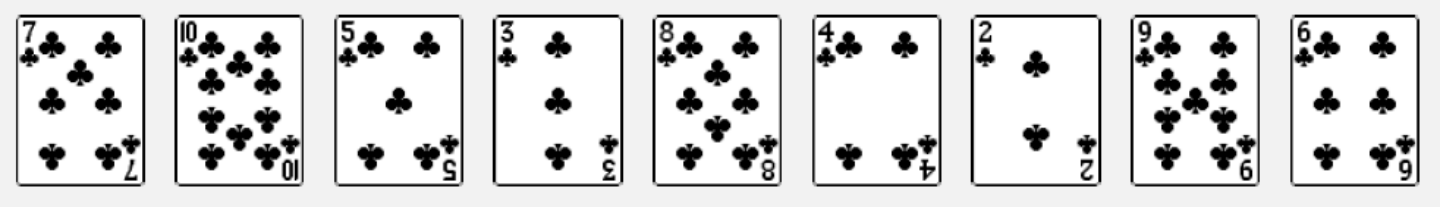
**- Selection sort** është një ndër algoritmet më të thjeshtë të sortimit që skanon nga e majta në të djathtë. Ai punon duke kërkuar elementin më të vogël në pjesën e mbetur të array-it dhe duke e vendosur në pozicionin e duhur. Rregullat:

- Elementet në të majtë të (përfshirë) janë të fiksuara dhe të rradhitura nga më i vogli, duke u rritur.
- Asnjë element në të djathtë të nuk është më i vogël sesa cilido element në të majtë të.



Në çdo hap:

- Për secilin pozicion i, gjejmë index-in e minimumit (min) në pjesën e djathtë.
- Bëjmë swap midis elementit në i dhe elementit në min.



**Metodat ndihmëse:**

- **less()** – krahason dy vlera (kompleksitet N).
- **exch()** – bën ndërrimin (edhe kjo N).

Selection sort ka dy procese: krahasimin **less()** dhe swap **exch(),** të dyja kanë kompleksitet N → pra gjithsej **$N^2$**.

**Karakteristikat:**

- Nuk është algoritëm i mirë, se ka gjithmonë $N^2$ krahasime, pavarësisht nëse array është i sortuar apo jo.
- Ka shumë pak swap-e (vetëm N), por shumë krahasime.
- Në trace: pjesa bold ekzekutohet, pjesa gri nuk ekzekutohet.

**- Insertion sort** funksionon ndryshe nga Selection. Në vend se të kërkojë minimumin për çdo pozicion, ai merr një element dhe e inserton në pozicionin e duhur te pjesa e majtë e sortuar. Për çdo i:

- Krahason a[i] me elementët në të majtë.
- Nëse elementët në të majtë janë më të mëdhenj, bën **swap** duke lëvizur elementin majtas derisa të vendoset në vendin e duhur.

**Karakteristikat:**

- Nëse array është pothuajse i sortuar → **shumë i shpejtë** (rasti më i mirë: N krahasime, 0 swap).
- Nëse array është në rend zbritës → rasti më i keq me **$\frac{1}{2}N^2$ krahasime + $\frac{1}{2}N^2$ ndërrime**.
- Më praktik se selection sort në shumicën e rasteve, kompleksiteti kohor i insertion sort është më i mirë se selection sort.

**- Shell Sort (h-sort)** është një version i përmirësuar i insertion sort. Ideja është që elementët të mos lëvizin vetëm nga një pozicion, por me hapa më të mëdhenj **(h-distance)**. Kjo e bën algoritmin shumë më të shpejtë.

- Zgjidhet një **vlerë e madhe h** (p.sh. 13, 4…).
- Bëhet **h-sort**, që është si insertion sort por elementët krahasohen me distancë h.
- h zvogëlohet (p.sh. 13 → 4 → 1).
- Kur arrijmë h = 1 → bëjmë insertion sort final.

Shell sort ose h-sort ngjason me insertion sort por ndryshon te swap dhe merr vlera h.

**Performanca:**

- varet nga sekuenca e h-ve.
- Me sekuencën **3x + 1** → rasti më i mirë rreth **N log N**.
- Rasti më i keq zakonisht rreth **N^3/2** (më mirë se $N^2$).
- Më i shpejtë se selection dhe insertion sort për lista të mëdha.

**- Informacion shtesë:**

- Sortimi organizon të dhënat dhe na lejon t'i gjejmë më lehtë.
- Për të sortuar, objektet duhet të krahasohen me **compareTo()** (0, -1, 1).
- Selection sort → *shumë krahasime, pak swap-e, gjithmonë $N^2$.*
- Insertion sort → *i shpejtë për lista pothuajse të sortuara*.
- Shell sort → *version i përmirësuar i insertion sort, shumë më i shpejtë*.
- "Kur krahasojmë, i ka vlerën i+1" – rregulli i loops në trace.
- "Rregulli i parë në algoritme është të mos humbësh të dhënat" → swap duhet të ruajë vlerat gjatë ndërrimit.

# 6. Merge Sort

- Mergesort dhe Quicksort janë dy ndër algoritmet klasike të sortimit dhe konsiderohen komponentë kritikë në infrastrukturën kompjuterike moderne.

- Quicksort është vlerësuar si një nga 10 algoritmet më të rëndësishme të shekullit XX.
- Mergesort është themelor për sisteme që kërkojnë **stabilitet** dhe **performancë të garantuar O(N log N)**.

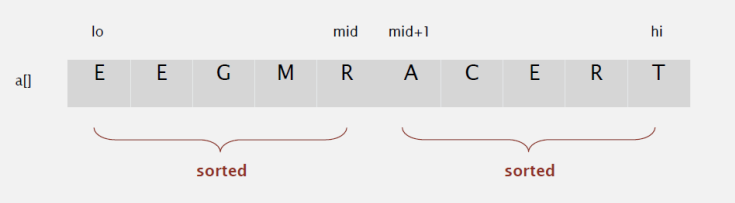
- Në këtë material fokusi është tek **Mergesort** (Top-Down & Bottom-Up) dhe pastaj prezantohet **Timsort**, algoritmi modern i gjuhëve Python, Java 7+ dhe Android. Mergesort punon sipas parimit **divide** and **conquer**:

1. **Nda** array-in në dy gjysma.
2. **Sorto** secilën gjysmë në mënyrë rekursive.
3. **Bashko** dy gjysmat e sortuara për të marrë rezultatin final.

Procesi i bashkimit quhet **merge**. Shembull për të bashkuar dy gjysma të sortuara:

**E E G M R** | **A C E R T** dhe pas bashkimit: **A C E E E G M R R T**

Mergesort gjithmonë prodhon një *array të sortuar në mënyrë të qëndrueshme (stable)*.




**- lo** është **indeksi i fillimit** të pjesës së vargut që po shqyrtojmë. Kur bëhet mergimi, lo tregon se ku fillon nën-vargu i parë i sortuar. Pra ai gjithmonë i referohet elementit të parë të një pjese që duam ta bashkojmë.

**- mid** është **pika e mesit** (apo fundi i nën-vargut të majtë), e cila ndan vargun në **dy nën-vargje**:

- nën-vargu i majtë: nga lo deri në mid
- fillimi i nën-vargut të djathtë: nga mid + 1 deri në hi

**- hi** është **indeksi i fundit** i pjesës së vargut që po shqyrtojmë. Kjo tregon fundin e nën-vargut të djathtë.

Gjatë bashkimit:

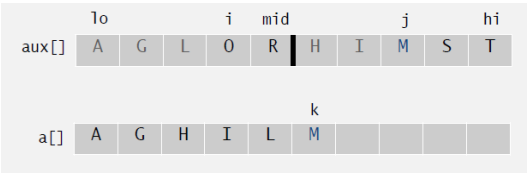
- pjesa **a[lo … mid]** është tashmë e **sortuar**
- pjesa **a[mid+1 … hi]** është gjithashtu **e sortuar**

Qëllimi është t'i **bashkojmë këto dy nën-vargje të sortuara** në një varg të vetëm të sortuar që shkon nga **lo** deri te **hi**.

**- Merge në Java** kryen **bashkimin** e dy nën-array-ve të sortuar.

```
private static void merge(Comparable[] a, Comparable[] aux, int lo, int mid, int hi)
{
   for (int k = lo; k <= hi; k++)
      aux[k] = a[k];                                          copy

   int i = lo, j = mid+1;
   for (int k = lo; k <= hi; k++)
   {
      if      (i > mid)                 a[k] = aux[j++];
      else if (j > hi)                  a[k] = aux[i++];
      else if (less(aux[j], aux[i])) a[k] = aux[j++];
      else                              a[k] = aux[i++];      merge
   }
}
```




**- Implementimi i plotë i Mergesort (Top-Down)**

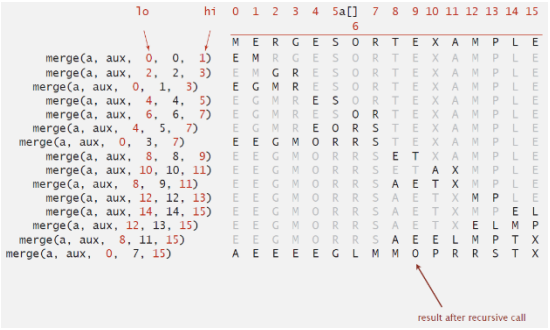
```
public class Merge
{
   private static void merge(...)
   { /* as before */ }

   private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
   {
      if (hi <= lo) return;
      int mid = lo + (hi - lo) / 2;
      sort(a, aux, lo, mid);
      sort(a, aux, mid+1, hi);
      merge(a, aux, lo, mid, hi);
   }

   public static void sort(Comparable[] a)
   {
      Comparable[] aux = new Comparable[a.length];
      sort(a, aux, 0, a.length - 1);
   }
}
```

**- Shembull i plotë (**Trace**):** Ky është procesi hap-pas-hapi për fjalën: **MERGESORTEXAMPLE,** pas shumë bashkimeve, rezultati final: A E E E E G L M M O P R R S T X

Ky trace konfirmon se mergesort është **stabil, i saktë dhe rekursiv**.




**- Analiza e kompleksitetit** të merge sort:

| **Tipi i kohës** | **Vlera** |
|---|---|
| **Worst case** | Θ(N log N) |
| **Best case** | Θ(N log N) (edhe nëse është pothuajse i sortuar) |
| **Average case** | Θ(N log N) |
| **Hapësira** | Θ(N) – kërkon array ndihmës |

Mergesort është optimal në krahasime, por nuk është optimal në hapësirë. Laptop ekzekuton $10^8$ krahasime/second. Kurse, në Superkompjuter ekzekuton $10^{12}$krahasime/second. Algorimet e mirë janë më mirë sesa superkompjuterët.

| | insertion sort ($N^2$) | | | mergesort (N log N) | | |
|---|---|---|---|---|---|---|
| computer | thousand | million | billion | thousand | million | billion |
| home | instant | 2.8 hours | 317 years | instant | 1 second | 18 min |
| super | instant | 1 second | 1 week | instant | instant | instant |

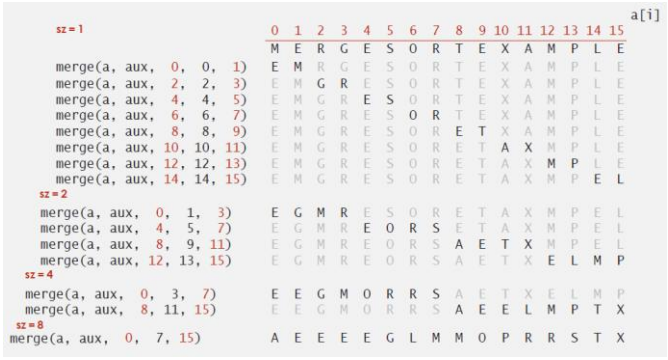
**- Përmirësime praktike:** Java-ja bën këto optimizime:

- Nëse pjesët janë tashmë të sortuara, mos bëj merge **if (!less(a[mid+1], a[mid])) return;**
- Cutoff në insertion sort për nën-array shumë të vogla (zakonisht < 7 elemente).
- Shmang kopjimin e panevojshëm të array-it në disa implementime.

```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
{
   if (hi <= lo) return;
   int mid = lo + (hi - lo) / 2;
   sort (a, aux, lo, mid);
   sort (a, aux, mid+1, hi);
   if (!less(a[mid+1], a[mid])) return;
   merge(a, aux, lo, mid, hi);
}
```

**- Bottom-Up Mergesort** version nuk përdor rekursion. Punon me madhësi në rritje:

- fillimisht merge subarray të madhësisë 1
- pastaj 2
- pastaj 4
- 8…

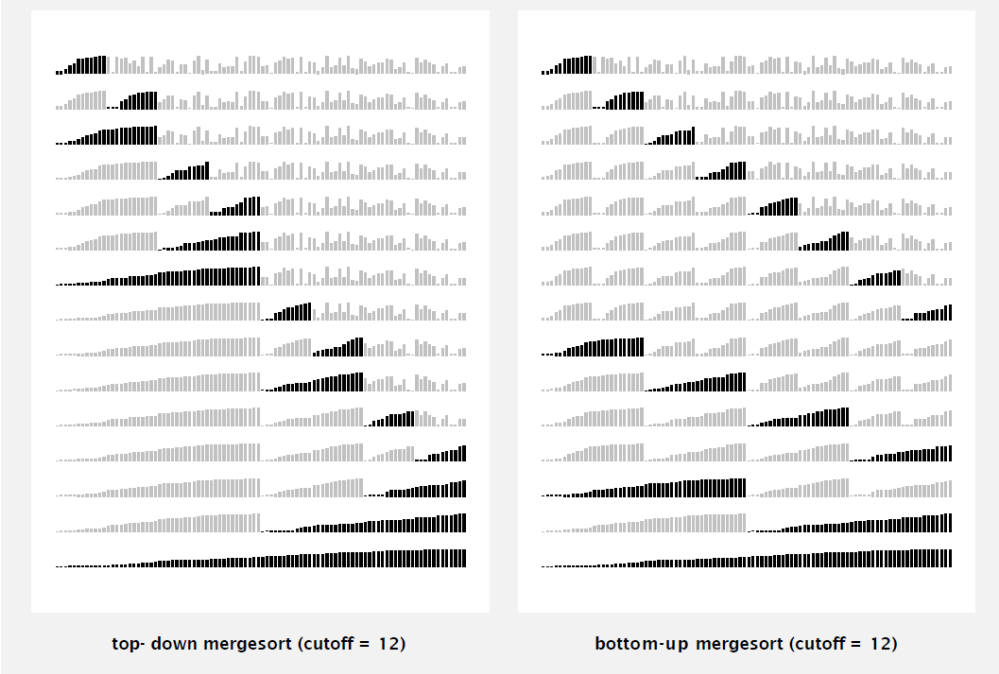



```
public class MergeBU
{
   private static void merge(...)
   {  /* as before */  }

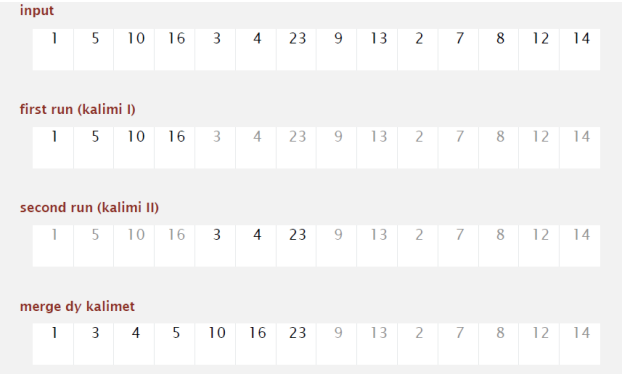
   public static void sort(Comparable[] a)
   {
      int N = a.length;
      Comparable[] aux = new Comparable[N];
      for (int sz = 1; sz < N; sz = sz+sz)
         for (int lo = 0; lo < N-sz; lo += sz+sz)
            merge(a, aux, lo, lo+sz-1, Math.min(lo+sz+sz-1, N-1));
   }
}
```

**Funksionon më ngadalë se top-down (~10%)** por është më i thjeshtë për sistemet iterative.

Dallimi në mes merge sort top-down dhe merge sort bottom-up në mënyrë vizuale:



**- Mergesort Natyral** nuk ndan array-n në gjysma, identifikon automatikisht "runs" (pjesë të sortuara natyrale) dhe i bashkon ato pjesë. Avantazh: shkon **O(N)** në array që janë pjesërisht të sortuar. Më pak kalime vs. ekstra krahasime për kalim për të idetifikuar kalimet.




**- TIMSORT** është një algoritëm modern që kombinohet nga mergesort natyral, binary insertion sort dhe optimizime të thella. Ka kohë lineare. Përdoret në **Python, Java 7+ Arrays.sort(), Android, Octave** dhe shumë biblioteka modern. Është **super i shpejtë** në array që kanë renditje ekzistuese.

**Stabiliteti i algoritmeve** i referohet aftësisë së një algoritmi për të ruajtur rendin fillestar të elementeve që kanë çelësa të njëjtë. Algoritmet “stable” si Mergesort, Insertion Sort, Bubble Sort dhe Counting/Radix Sort garantojnë që elementet me vlerë të barabartë të mbeten në të njëjtën renditje si në hyrje. Nga ana tjetër, algoritmet jostabile si Selection Sort, Shell Sort dhe Quicksort (në disa implementime) mund të shkatërrojnë rendin e mëparshëm.

**- Në Java, Comparator** përdoret për të sortuar objekte të klasave të personalizuara sipas kritereve të ndryshme**.** Për shembull, mund të sortosh një array studentësh sipas emrit ose sipas seksionit duke përdorur: **Arrays.sort(a, new Student.ByName());** ose **Arrays.sort(a, new Student.BySection());.** Çdo comparator implementon metodën compare(Student a, Student b), e cila përcakton rendin midis dy objekteve. Stabiliteti është veçanërisht i rëndësishëm kur kryhen sortime të shumëfishta njëra pas tjetrës.

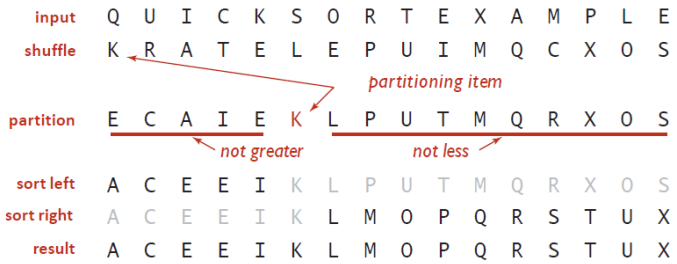
- Në përfundim, Mergesort është një nga algoritmet më të fuqishëm dhe të besueshëm për renditje. Ai ofron performancë të garantuar O(N log N), është i qëndrueshëm (stable) dhe përdoret gjerësisht në sisteme ku ruajtja e rendit të elementeve është e domosdoshme. Megjithatë, Mergesort nuk është optimal në përdorimin e hapësirës, pasi kërkon Θ(N) memorie shtesë. Versionet moderne si Timsort, që kombinojnë idetë e Mergesort dhe Insertion Sort, janë ndër algoritmet më të shpejtë aktualisht dhe janë pikërisht ato që Java dhe Python përdorin për sortimin e objekteve.

# 7. Quick Sort

Mergesort dhe Quicksort janë dy nga algoritmet më të rëndësishëm të sortimit në historinë e shkencave kompjuterike. Ato përdoren gjerësisht për shkak të efikasitetit të tyre, dhe studimi i tyre i thelluar ka bërë të mundur optimizimin e shumë sistemeve kompjuterike moderne. Quicksort madje vlerësohet si një nga 10 algoritmet më të mirë të shekullit të 20-të, falë shpejtësisë dhe thjeshtësisë së implementimit.

- **Mergesort** funksionon duke ndarë vazhdimisht vargun në dy gjysma deri sa të arrijë në elemente individuale. Më pas, këto pjesë të vogla bashkohen përsëri duke i radhitur. Me fjalë të thjeshta **e ndajmë problemin në pjesë të vogla → i zgjidhim → i bashkojmë**. Avantazhi i tij është se është gjithmonë stabil dhe gjithmonë O(n log n), pavarësisht nga renditja fillestare.

**-** **Quicksort** punon ndryshe nga Mergesort, plani bazë i tij është që së pari bën **shuffle** vargun dhe ai zgjedh një element si pikë reference, të quajtur **pivot**, dhe vendos rreth tij të gjithë elementet më të vegjël në të majtë dhe më të mëdhenjtë në të djathtë. Ky proces quhet **particionim**. Pas ndarjes, të dy pjesët sortohen përsëri në mënyrë rekursive. Pra, **zgjedh pivot → ndaj vargun në të majtë/djathtë → sorto pjesët**.




**-** Nëse vargu është tashmë pothuajse i renditur, Quicksort mund të bëhet shumë i ngadalshëm. Prandaj, fillimisht bëhet një **shuffle** (përzierje e rastësishme). Ky proces përdor një **pseudo-random generator**, i cili ndihmon që pivotët të mos bien gabimisht gjithmonë në pozicion të keq, duke garantuar performancë të mirë.

**-** Gjatë particionimit përdoren dy "pointera": **i** dhe **j**.

- Nëse **a[i] < a[lo]** → i lëviz në të djathtë (i++)
- Nëse **a[j] < a[lo]** → j lëviz në të majtë (j--)
- Nëse të dy kushtet nuk plotësohen, do të thotë që kemi gjetur dy vlera që janë në anët e gabuara dhe këto dy vlera shkëmbehen (*swap*) **a[i] me a[j].**

Kjo vazhdon deri sa i dhe j takohen. Në fund, pivot shkëmbehet me elementin në pozicionin j, dhe vargu ndahet në dy pjesë që sortohen veçmas.

Quicksort është **in-place,** sepse nuk përdor memorje shtesë për vargje ndihmëse si Mergesort. Nuk është stabil, sepse gjatë swap-eve mund të shkëmbehen dy elemente të njëjtë dhe të humbet renditja e tyre fillestare. Rasti më i mirë dhe më i keq i Quicksort

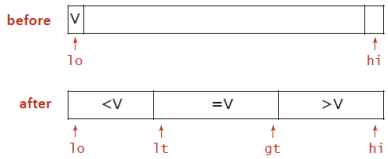
- **Rasti më i mirë:** Pivot ndan vargun saktë në dy pjesë të barabarta → O(n log n).
- **Rasti më i keq:** Pivot zgjedhet shumë → O($n^2$).

Kjo është arsyeja pse **shuffle** është kaq i rëndësishëm.

**- Problemi me çelësat e dyfishtë (***duplicate keys***):** kur ka shumë elemente të njëjtë, Quicksort mund të bëhet shumë i ngadalshëm nëse particioni nuk ndalet tek elementet që janë të barabartë me pivotin.

**-** Kur ka shumë vlera të përsëritura, përdoret një version i Quicksort i quajtur **3-way partitioning dijkstra**, që e ndan vargun në tre pjesë:

- vlera **më të vogla** se pivot
- vlera **të barabarta** me pivot
- vlera **më të mëdha** se pivot




Këtu përdoren tre pointerë: **lt**, **i**, dhe **gt**. Ky version është shumë më efikas për vargje me shumë duplikate.

- **(a[i] < v):** exchange **a[lt]** with **a[i];** increment both **lt** and **i.**
- **(a[i] > v):** exchange **a[gt]** with **a[i];** decrement **gt.**
- **(a[i] == v) :**increment **i.**







**-** Quicksort konsiderohet algoritmi më i shpejtë në botë për shumicën e rasteve praktike, për shkak të mënyrës së tij intelligente të particionimit dhe përdorimit të pivotit. Megjithatë, ai ka rreziqe nëse nuk bëhet shuffle ose kur ka shumë duplikate. Për këto raste përdoret 3-way partitioning, i cili zgjidh problemin në mënyrë të shkëlqyer. Ndërkohë, Mergesort është më i qëndrueshëm dhe i parashikueshëm, por kërkon memorie shtesë.

**Mësimi 1.** Good algorithms are better than supercomputers.
**Mësimi 2.** Great algorithms are better than good ones.

| | inplace? | stable? | best | average | worst | remarks |
|---|---|---|---|---|---|---|
| selection | ✓ | | $\frac{1}{2} N^2$ | $\frac{1}{2} N^2$ | $\frac{1}{2} N^2$ | $N$ exchanges |
| insertion | ✓ | ✓ | $N$ | $\frac{1}{4} N^2$ | $\frac{1}{2} N^2$ | use for small $N$ or partially ordered |
| shell | ✓ | | $N \log_3 N$ | ? | $c N^{3/2}$ | tight code; subquadratic |
| merge | | ✓ | $\frac{1}{2} N \lg N$ | $N \lg N$ | $N \lg N$ | $N \log N$ guarantee; stable |
| timsort | | ✓ | $N$ | $N \lg N$ | $N \lg N$ | improves mergesort when preexisting order |
| quick | ✓ | | $N \lg N$ | $2 N \ln N$ | $\frac{1}{2} N^2$ | $N \log N$ probabilistic guarantee; fastest in practice |
| 3-way quick | ✓ | | $N$ | $2 N \ln N$ | $\frac{1}{2} N^2$ | improves quicksort when duplicate keys |
| ? | ✓ | ✓ | $N$ | $N \lg N$ | $N \lg N$ | holy sorting grail |

# 8. Priority Queues

**- Collections** janë struktura të të dhënave që përdoren për të ruajtur një grup elementesh. Ato organizojnë të dhënat në mënyra të ndryshme, varësisht nga operacionet që duam të bëjmë. Për shembull, një *stack* përdoret kur duam të heqim gjithmonë elementin e fundit të shtuar, ndërsa një *queue* kur duam të heqim të parin. Priority Queue ndryshon nga këto të dyja sepse heq gjithmonë **elementin me prioritetin më të madh (ose të vogël)**.

| data type | key operations | data structure |
|---|---|---|
| **stack** | PUSH, POP | *linked list, resizing array* |
| **queue** | ENQUEUE, DEQUEUE | *linked list, resizing array* |
| **priority queue** | INSERT, DELETE-MAX | *binary heap* |
| **symbol table** | PUT, GET, DELETE | *BST, hash table* |
| **set** | ADD, CONTAINS, DELETE | *BST, hash table* |

- Priority Queue (PQ) është një strukturë e të dhënave ku:

- Mund të fusim elemente (insert)
- Mund të heqim elementin me prioritetin më të madh (delete-max)

Kjo e bën PQ më të avancuar se Stack dhe Queue, sepse nuk punon sipas rregullave *FIFO* ose *LIFO*, por sipas **vlerës** (*prioritetit*). PQ përdoret për algoritme shumë të rëndësishme si Dijkstra, Prim, simulime, kodim Huffman, inteligjencë artificiale, planifikim, rrjete kompjuterike etj.

Për të funksionuar, elementet duhet të jenë **Comparable**, që domethënë duhet të kenë një mënyrë për t'u krahasuar (p.sh. numra, string, etj.).

```
public class MaxPQ<Key extends Comparable<Key>>
```

| | | |
|---|---|---|
| | `MaxPQ()` | *create an empty priority queue* |
| | `MaxPQ(Key[] a)` | *create a priority queue with given keys* |
| `void` | `insert(Key v)` | *insert a key into the priority queue* |
| `Key` | `delMax()` | *return and remove the largest key* |
| `boolean` | `isEmpty()` | *is the priority queue empty?* |
| `Key` | `max()` | *return the largest key* |
| `int` | `size()` | *number of entries in the priority queue* |

**-** Për shkak se na duhet të punojmë me të dhëna ku **renditja sipas prioritetit** është thelbësore. PQ përdoret në shumë fusha:

- **Algoritmet e grafikëve** (Dijkstra, Prim)
- **Simulime me evente**
- **Balancim ngarkese**
- **Procesim statistikash**
- **Caching në web**
- **Sisteme operative (interrupt handling)**
- **Inteligjencë artificiale (A* search)**
- **Siguri / Spam filtering**

| implementation | time | space |
|---|---|---|
| **sort** | $N \log N$ | $N$ |
| **elementary PQ** | $M\,N$ | $M$ |
| **binary heap** | $N \log M$ | $M$ |
| **best in theory** | $N$ | $M$ |

**order of growth për të gjetur M-në më të madhe në një stream prej N elementeve**

PQ është një mjet i fuqishëm që përgjithëson Stack, Queue dhe Randomized Queue.

**- Implementimet e Priority Queue**

**a) Array i pasortuar**

- Insert = shumë i shpejtë (O(1))
- delMax = shumë i ngadalshëm (O(N))
- max = O(N)

```
public class UnorderedArrayMaxPQ<Key extends Comparable<Key>>
{
   private Key[] pq;    // pq[i] = ith element on pq
   private int N;       // number of elements on pq

   public UnorderedArrayMaxPQ(int capacity)
   {  pq = (Key[]) new Comparable[capacity];  }      <- no generic array creation

   public boolean isEmpty()
   {  return N == 0; }

   public void insert(Key x)
   {  pq[N++] = x;  }

   public Key delMax()
   {
      int max = 0;
      for (int i = 1; i < N; i++)
         if (less(max, i)) max = i;                  <- less() and exch() similar to sorting methods (but don't pass pq[])
      exch(max, N-1);
      return pq[--N];                                <- should null out entry to prevent loitering
   }
}
```

**b) Array i sortuar**

- Insert = shumë i ngadalshëm (O(N))
- delMax = shumë i shpejtë (O(1))
- max = O(1)

| implementation | insert | del max | max |
|---|---|---|---|
| unordered array | 1 | *N* | *N* |
| ordered array | *N* | 1 | 1 |
| goal | log *N* | log *N* | log *N* |

order of growth për running time për priority queue me N elemente

| operation | argument | return value | size | contents (unordered) | contents (ordered) |
|---|---|---|---|---|---|
| insert | P | | 1 | P | P |
| insert | Q | | 2 | P Q | P Q |
| insert | E | | 3 | P Q E | E P Q |
| remove max | | Q | 2 | P E | E P |
| insert | X | | 3 | P E X | E P X |
| insert | A | | 4 | P E X A | A E P X |
| insert | M | | 5 | P E X A M | A E M P X |
| remove max | | X | 4 | P E M A | A E M P |
| insert | P | | 5 | P E M A P | A E M P P |
| insert | L | | 6 | P E M A P L | A E L M P P |
| insert | E | | 7 | P E M A P L E | A E E L M P P |
| remove max | | P | 6 | E M A P L E | A E E L M P |

Sekuenca e operacioneve në një priority queue

**c) Binary Heap (më i përdoruri)**

- Insert = O(log N)
- delMax = O(log N)
- max = O(1)

Binary Heap është implementimi standard sepse është **më efikasi**.

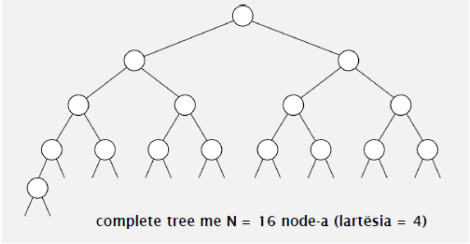
**-** Binary Heap përfaqësohet nga një **array**, por konceptualisht është një **peme binare e plotë** (complete binary tree). Ka dy rregulla:

1. **Rregulli i plotësisë (complete tree)** Çdo nivel është i plotë, përveç nivelit të fundit. Treguesi starton tek 1, mirret nyja dhe renditet në nivelin përkatës. Nuk ka linka medoemos.
2. **Heap Order Property** Prindi ka gjithmonë vlerë më të madhe se fëmijët e tij.

Kjo e garanton që elementi më i madh është gjithmonë në **rrënjë → a[1]**.

**Vetitë:** Lartësia e complete tree me N node-a është lg N.

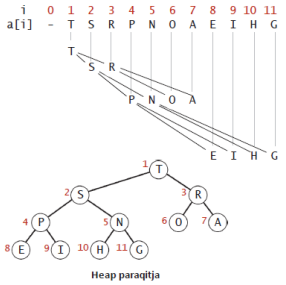
**Vërtetim:** Lartësia rritet vetëm kur N është në fuqi të 2.



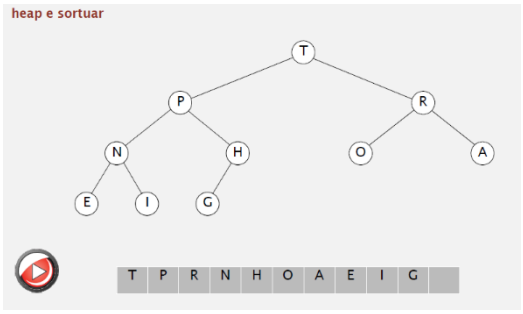
complete tree me N = 16 node-a (lartësia = 4)

Mund të përdoren treguesit e array për të lëvizur nëpër tree.

- Prindi i nyjës k është k/2.
- Fëmijët e nyjës k janë 2k dhe 2k+1.



Heap paraqitja




**Lëvizjet:**

- **swim (***noton lart***)** – kur futim një element të ri
- **sink (***zhytet poshtë***)** – kur heqim max

Këto procese ruajnë rregullin e heap-it.

Kur fusim një element të ri:

- E vendosim në fund të pemës (fundin e array-t)
- E krahasojmë me prindin
- Nëse është më i madh → i ndërrojmë vendet
- Përsërisim derisa rregulli i heap-it të plotësohet

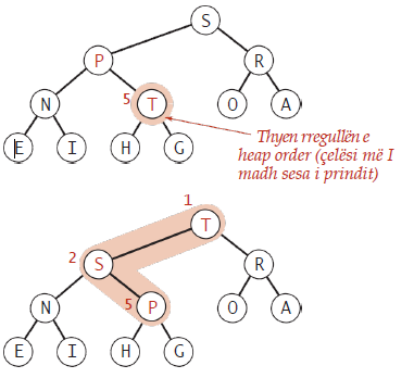
Kjo quhet **swim**, dhe kërkon maksimumi **1+lg N** krahasime.

Kur heqim elementin më të madh (rrënjën):

- Ndërrojmë rrënjën me elementin e fundit
- E heqim elementin e fundit (që është max)
- Elementi i ri në rrënjë mund të jetë më i vogël se fëmijët, prandaj:
  - Krahasohet me fëmijët
  - Ndërron vendin me fëmijën më të madh
  - Procesi vazhdon derisa të rivendoset rregulli

Kjo quhet **sink**, me kosto **2 lg N** krahasime.

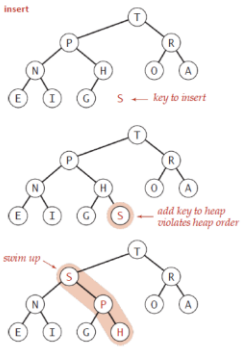
```
private void swim(int k)
{
   while (k > 1 && less(k/2, k))
   {
      exch(k, k/2);
      k = k/2;
   }
}
```
parent of node at k is at k/2



**- Operacione kryesore në një heap (*swim dhe sink*):** Kur një element futet në heap, ai fillimisht vendoset në fund të array-t dhe mund të prishë rregullin e heap-order. Për ta rregulluar këtë, elementit i lejohet të "noton lart", proces i quajtur *swim*. Swim krahason nyjën me prindin e saj dhe, nëse prindi është më i vogël, ato shkëmbehen. Kjo vazhdon derisa elementi të arrijë pozicionin e duhur ose majën e heap-it. E kundërta ndodh me metodën *sink*, që përdoret zakonisht pas fshirjes së elementit maksimal. Një nyjë lëviz poshtë duke u shkëmbyer me fëmijën më të madh derisa të rikthehet rregulli i heap-order.

**- Insertimi dhe fshirja e elementit maksimum**: në një max-heap vendos një element të ri në fund të array-t dhe pastaj thërret *swim*. Ky proces ka kosto maksimale rreth log N krahasimesh. Ndërsa fshirja e elementit maksimal heq rrënjën e heap-it, shkëmben elementin e parë me të fundit, dhe pastaj kryen *sink* për të zbritur elementin në pozicionin e duhur. Ky operacion gjithashtu ka kosto rreth log N.

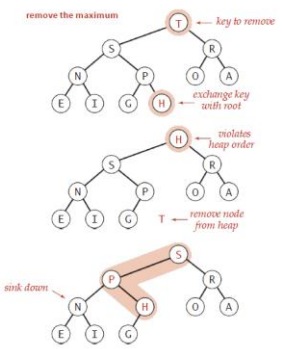
```
public void insert(Key x)
{
   pq[++N] = x;
   swim(N);
}
```




```
public Key delMax()
{
   Key max = pq[1];
   exch(1, N--);
   sink(1);

}  pq[N+1] = null;
   return max;
```

prevent loitering




```
public class MaxPQ<Key extends Comparable<Key>>
{
   private Key[] pq;
   private int N;

   public MaxPQ(int capacity)
   {  pq = (Key[]) new Comparable[capacity+1];  }

   public boolean isEmpty()
   {  return N == 0;  }
   public void insert(Key key)
   public Key delMax()
   {  /* see previous code */  }

   private void swim(int k)
   private void sink(int k)
   {  /* see previous code */  }

   private boolean less(int i, int j)
   {  return pq[i].compareTo(pq[j]) < 0;  }
   private void exch(int i, int j)
   {  Key t = pq[i]; pq[i] = pq[j]; pq[j] = t;  }
}
```

fixed capacity (for simplicity)

PQ ops

heap helper functions

array helper functions

**- Implementimi i heap-it në Java**: një heap zakonisht implementohet me një array ku pozicioni 1 përdoret si rrënja. Funksionet ndihmëse si less() dhe exch() thjeshtojnë krahasimin dhe shkëmbimin. Klasa MaxPQ përfshin metodat bazë: insert, delMax, isEmpty, si dhe metodat private sink dhe swim. Kjo implementim lejon ruajtjen dhe manipulimin efikas të prioriteteve.

**- Kostoja (order-of-growth) e strukturave të ndryshme të priority queue**: struktura të ndryshme të priority queue kanë kosto të ndryshme për operacionet kryesore. Një array i pa-rregulluar mund të fusë elementin shpejt, por kërkimi i maksimumit është i ngadaltë. Një array i renditur ka insertim shumë të ngadalshëm, por fshirjen e maksimumit shumë të shpejtë. Heap-et ofrojnë kompromisin ideal: të dy operacionet bëhen në kohë logaritmike. Strukturat më të avancuara si Fibonacci heap përmirësojnë disa operacione, por janë më të ndërlikuara për implementim praktik.

| implementation | insert | del max | max |
|---|---|---|---|
| unordered array | 1 | $N$ | $N$ |
| ordered array | $N$ | 1 | 1 |
| binary heap | $\log N$ | $\log N$ | 1 |
| d-ary heap | $\log_d N$ | $d \log_d N$ | 1 |
| Fibonacci | 1 | $\log N$ † | 1 |
| Brodal queue | 1 | $\log N$ | 1 |
| impossible | 1 | 1 | 1 |

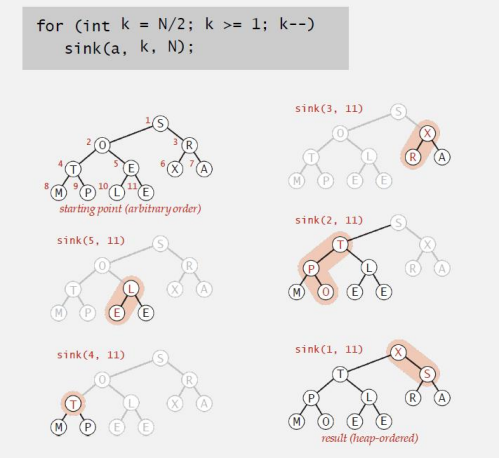
**- Immutability në Java**: një tip i dhënash *immutable* nuk mund të ndryshohet pasi është krijuar. Shembuj janë String, Integer, Double, dhe klasa të tjera të projektimit funksional. Këto tipe janë të dobishme sepse thjeshtojnë debugging, janë më të sigurta dhe lehtësojnë programimin paralel. Nga ana tjetër, tipat mutable si arrays, counter ose StringBuilder mund të modifikohen pas krijimit. Në implementim, një klasë immutable bën kopje mbrojtëse të të dhënave dhe deklaron variablat si private dhe final. Kjo është e vlefshme në PQ sepse:

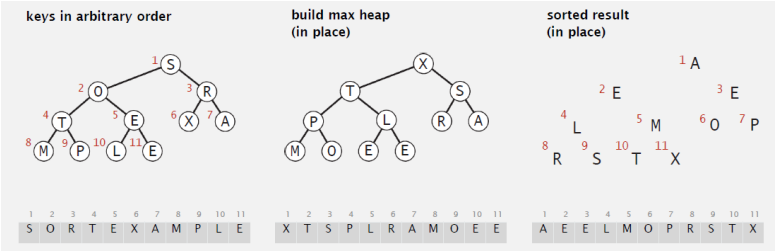
- E bën kodin më të sigurt
- E thjeshton debugging
- Lehtëson programimin paralel
- Siguron korrektësi në krahasime, që janë jetike për PQ

**- Heapsort dhe procesi i ndërtimit të heap-it**: heapsorti përdor heap-in për të bërë sortimin në vend (*in-place*). Procesi ka dy faza:

- ndërtimi i një max-heap nga array i pa-renditur, i cili zakonisht realizohet me teknikën *bottom-up*, duke thirrur sink për çdo nyjë jo-gjethe
- vazhdimisht shkëmbimi i rrënjës me elementin e fundit dhe ulja e saj poshtë me sink dhe rezultati final është një array i renditur në mënyrë rritëse.

Konstruktimi i heap-it merr O(N) kohë, ndërsa faza e sortimit merr O(N log N).




| | inplace? | stable? | best | average | worst | remarks |
|---|---|---|---|---|---|---|
| selection | ✓ | | $\frac{1}{2} N^2$ | $\frac{1}{2} N^2$ | $\frac{1}{2} N^2$ | $N$ exchanges |
| insertion | ✓ | ✓ | $N$ | $\frac{1}{4} N^2$ | $\frac{1}{2} N^2$ | use for small $N$ or partially ordered |
| shell | ✓ | | $N \log_3 N$ | ? | $c N^{3/2}$ | tight code; subquadratic |
| merge | | ✓ | $\frac{1}{2} N \lg N$ | $N \lg N$ | $N \lg N$ | $N \log N$ guarantee; stable |
| timsort | | ✓ | $N$ | $N \lg N$ | $N \lg N$ | improves mergesort when preexisting order |
| quick | ✓ | | $N \lg N$ | $2 N \ln N$ | $\frac{1}{2} N^2$ | $N \log N$ probabilistic guarantee; fastest in practice |
| 3-way quick | ✓ | | $N$ | $2 N \ln N$ | $\frac{1}{2} N^2$ | improves quicksort when duplicate keys |
| heap | ✓ | | $N$ | $2 N \lg N$ | $2 N \lg N$ | $N \log N$ guarantee; in-place |
| ? | ✓ | ✓ | $N$ | $N \lg N$ | $N \lg N$ | holy sorting grail |

# 9. Tabelat simbolike (*Symbol Tables*)

**- Symbol Table (*ST*)** është një **abstraksion key–value pair**, ku secilit **çelës (*key*)** i asociohet **një vlerë (*value*)**.

Funksioni kryesor i një symbol table është:

- të **ruajë** çifte çelës–vlerë,
- të mundësojë **kërkim efikas** të vlerës për një çelës të dhënë.

Operacionet bazë janë:

- **Insert**: futja e një vlere me çelës të specifikuar.
- **Search (*Get*)**: gjetja e vlerës që korrespondon me një çelës.
- **Delete**

**-** Një shembull klasik i symbol table është **DNS (Domain Name System)**:

- **Çelësi**: emri i domain-it
- **Vlera**: IP adresa përkatëse

| Domain name | IP address |
|---|---|
| www.cs.princeton.edu | 128.112.136.11 |
| www.princeton.edu | 128.112.128.15 |
| www.yale.edu | 130.132.143.21 |
| www.harvard.edu | 128.103.060.55 |
| www.simpsons.com | 209.052.165.60 |

- Symbol tables përdoren gjerësisht në informatikë:

| aplikimi | qëllimi për search | çelësi | vlera |
|---|---|---|---|
| dictionary | find definition | word | definition |
| book index | find relevant pages | term | list of page numbers |
| file share | find song to download | name of song | computer ID |
| financial account | process transactions | account number | transaction details |
| web search | find relevant web pages | keyword | list of page names |
| compiler | find properties of variables | variable name | type and value |
| routing table | route Internet packets | destination | best route |
| DNS | find IP address | domain name | IP address |
| reverse DNS | find domain name | IP address | domain name |
| genomics | find markers | DNA string | known positions |
| file system | find file on disk | filename | location on disk |

- Symbol tables njihen edhe si maps, fjalorë dhe array associative Ato gjeneralizojnë array-t klasike, sepse:

- çelësat **nuk kufizohen** në intervalin 0 … N-1,
- çelësat mund të jenë strings, objekte, etj.

**-** Një API tipik për symbol table përmban:

- put(Key key, Value val) – fut çiftin key–value
- get(Key key) – kthen vlerën ose null
- contains(Key key) – kontrollon ekzistencën
- delete(Key key) – fshin çelësin
- isEmpty() – kontrollon nëse tabela është bosh
- size() – numri i çifteve
- keys() – të gjithë çelësat

**- Konventa të rëndësishme**

- Vlerat nuk duhet të jenë null
- get() kthen null nëse çelësi nuk ekziston
- put() mbishkruan vlerën ekzistuese
- contains() implementohet lehtë **return get(key) != null;**

**Lazy delete** fshirja realizohet duke vendosur: **put(key, null);**

**- Çelësat dhe vlerat**

- **Vlerat**: cilido tip gjenerik
- **Çelësat** mund të përdorin compareTo(), equals() ose hashCode() (për scramble)

- Çelësat duhet të jenë **immutable (***të pandryshueshëm***)**, p.sh: Integer, Double, String, File, kurse mutable (jo të përshtatshëm): StringBuilder, URL, arrays.

- Në Java, equals() duhet të plotësojë:

- **Refleksive**: x.equals(x) == true
- **Simetrike**: x.equals(y) == y.equals(x)
- **Transitive**: x.equals(y) dhe y.equals(z) → x.equals(z)
- **Jo-null**: x.equals(null) == false

**- Implementimi me Linked List (***Sequential Search***),**

- **Struktura:** Linked list e pasortuar me çifte key–value
- **Search:** skanohet e gjithë lista → **O(N)**
- **Insert:** nëse nuk ka match, shtohet në fillim, → **O(N)**

**- Implementimi me Binary Search (***Array i sortuar***)**

- **Struktura:** Array i sortuar i çifteve key–value
- **Search:** përdoret binary search kompleksiteti: → **O(logN)**
- **Problemi:** Insert dhe delete kërkojnë zhvendosje të elementeve → N

| ST implementation | guarantee | | average case | | key interface |
|---|---|---|---|---|---|
| | search | insert | search hit | insert | |
| sequential search (unordered list) | $N$ | $N$ | $N / 2$ | $N$ | `equals()` |
| binary search (ordered array) | $\log N$ | $N$ | $\log N$ | $N / 2$ | `compareTo()` |

**- API për Symbol Table të sortuar,** operacionet shtesë:

- min(), max()
- floor(), ceiling()
- rank(), select()
- deleteMin(), deleteMax()
- keys(lo, hi) – range query

| | sequential search | binary search |
|---|---|---|
| search | $N$ | $\log N$ |
| insert / delete | $N$ | $N$ |
| min / max | $N$ | 1 |
| floor / ceiling | $N$ | $\log N$ |
| rank | $N$ | $\log N$ |
| select | $N$ | 1 |
| ordered iteration | $N \log N$ | $N$ |

- Symbol tables janë **ADT shumë të rëndësishme** për kërkim dhe ruajtje të dhënash. Ato përdoren gjerësisht në:

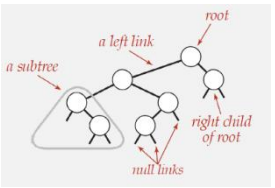
- sisteme operative,
- kompajlerë,
- web,
- databaza,
- aplikacione reale.

Zgjedhja e implementimit ndikon drejtpërdrejt në **performancë**. Për përdorim praktik dhe efikas, nevojiten struktura më të avancuara si **BST të balancuara** ose **Hash Tables**.

| implementation | guarantee | | | average case | | | ordered ops? | key interface |
|---|---|---|---|---|---|---|---|---|
| | search | insert | delete | search hit | insert | delete | | |
| **sequential search (unordered list)** | $N$ | $N$ | $N$ | $\frac{1}{2} N$ | $N$ | $\frac{1}{2} N$ | | `equals()` |
| **binary search (ordered array)** | $\lg N$ | $N$ | $N$ | $\lg N$ | $\frac{1}{2} N$ | $\frac{1}{2} N$ | ✓ | `compareTo()` |
| **BST** | $N$ | $N$ | $N$ | $1.39 \lg N$ | $1.39 \lg N$ | $\sqrt{N}$ | ✓ | `compareTo()` |
| **goal** | $\log N$ | $\log N$ | $\log N$ | $\log N$ | $\log N$ | $\log N$ | ✓ | `compareTo()` |

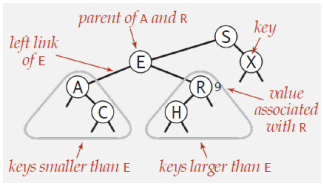
# 10. Binary Search Tree (*BST*)

**- Binary Search Tree (*BST*)** është një **binary tree** i veçantë që respekton një **rregull renditjeje simetrike** të çelësave. Një binary tree mund të jetë:




- i zbrazët (null), ose
- i përbërë nga një nyjë (node) që ka **dy subtree të veçanta**: të majtë dhe të djathtë.

- Në një BST, **çdo nyjë ka një çelës unik**, dhe zbatohet rregulli kryesor:




- Çdo çelës në subtree të majtë është **më i vogël** se çelësi i nyjës.
- Çdo çelës në subtree të djathtë është **më i madh** se çelësi i nyjës.

Ky rregull vlen rekursivisht për çdo subtree dhe është themeli i funksionimit të BST.

BST përfaqësohet nga një referencë te **root node**. Çdo **Node** përmban 4 fusha:

- një **Key** (çelës) dhe një **Value** (vlerë),
- një referencë te **subtree i majtë** dhe një referencë te **subtree i djathtë**.

Nëse një nyjë nuk ka fëmijë majtas ose djathtas, referenca është **null**. Këto quhen **null links**. Në Java, **Key dhe Value** janë tipe gjenerike**,** kurse **Key** është Comparable, që të mund të përdoret metoda compareTo().

**-** Operacioni **search (*get*)** përdoret për të gjetur vlerën e një çelësi.

```
public Value get(Key key)
{
   Node x = root;
   while (x != null)
   {
      int cmp = key.compareTo(x.key);
      if      (cmp  < 0) x = x.left;
      else if (cmp  > 0) x = x.right;
      else if (cmp == 0) return x.val;
   }
   return null;
}
```

- Nëse çelësi është **më i vogël**, shkohet majtas.
- Nëse është **më i madh**, shkohet djathtas.
- Nëse është **i barabartë**, search është i suksesshëm (search hit).
- Nëse është **null**, atëherë insert.

Nëse gjatë kërkimit arrihet një nyjë **null**, atëherë çelësi **nuk ekziston** në BST dhe kthehet null.

**Çmimi (*kostoja*)**:

- Numri i krahasimeve është baraz me **1 + thellësia e nyjës** në dru.
- Rasti mesatar: ~1.39 log N
- Rasti më i keq: N

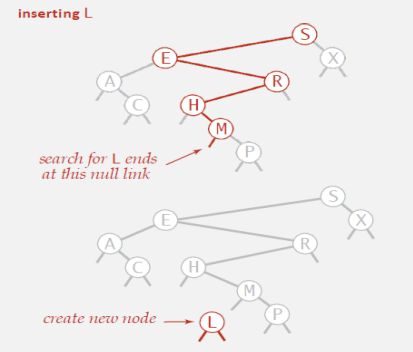
Operacioni **insert (put)** shërben për shtimin e një çelësi të ri me vlerën përkatëse, ose përditësimin e vlerës nëse çelësi ekziston. Procesi, kryhet search për çelësin, dhe:

- Nëse gjendet → vlera zëvendësohet.
- Nëse nuk gjendet → krijohet një **nyjë e re** në vendin ku haset një lidhje null.

Insert-i realizohet në mënyrë **rekursive** dhe gjatë kthimit:

- rifreskohen lidhjet,
- përditësohet count (numri i nyjëve në subtree).

Kostoja është **1 + thellësia e nyjës**.




```
public void put(Key key, Value val)
{  root = put(root, key, val);  }

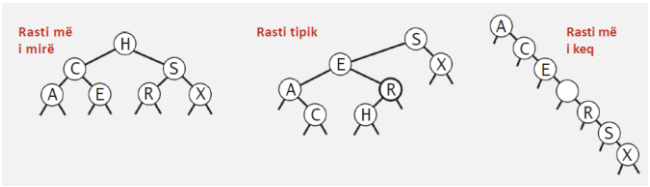
private Node put(Node x, Key key, Value val)
{
   if (x == null) return new Node(key, val);
   int cmp = key.compareTo(x.key);
   if      (cmp  < 0)
      x.left  = put(x.left,  key, val);
   else if (cmp  > 0)
      x.right = put(x.right, key, val);
   else if (cmp == 0)
      x.val = val;
   return x;
}
```

recursive
read care

Forma e drurit varet nga renditja e insertimeve:

- **Rasti më i mirë**: BST i balancuar → lartësi log N
- **Rasti tipik**: afërsisht i balancuar
- **Rasti më i keq**: çelësa të renditur → BST shndërrohet në listë → lartësi N

Sa më i thellë të jetë druri, aq **më i ngadalshëm** bëhet BST.




| implementation | guarantee | | average case | | operations on keys |
|---|---|---|---|---|---|
| | search | insert | search hit | insert | |
| sequential search (unordered list) | $N$ | $N$ | $\frac{1}{2} N$ | $N$ | `equals()` |
| binary search (ordered array) | $\lg N$ | $N$ | $\lg N$ | $\frac{1}{2} N$ | `compareTo()` |
| BST | $N$ | $N$ | $1.39 \lg N$ | $1.39 \lg N$ | `compareTo()` |

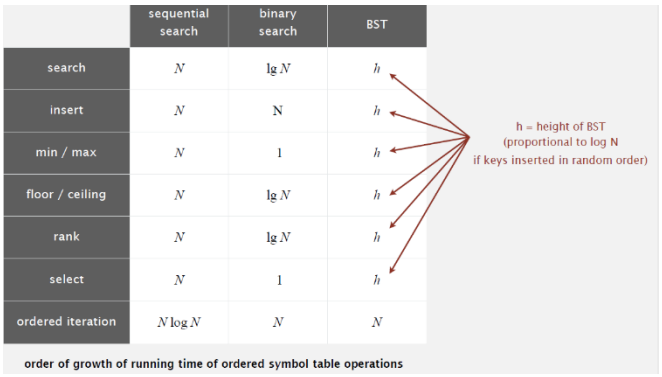
**- Minimum, Maximum, Floor dhe Ceiling**

- **Minimum**: çelësi më i vogël, gjendet duke shkuar gjithmonë majtas.
- **Maximum**: çelësi më i madh, gjendet duke shkuar gjithmonë djathtas.
- **Floor (*kufiri i poshtëm*)** çelësi më i madh që është **≤ k**. Rastet:
  - k == key → floor është vetë k.
  - k < key → floor ndodhet në subtree të majtë.
  - k > key → floor është në subtree të djathtë (nëse ekziston), përndryshe është nyja aktuale.
- **Ceiling (*kufiri i lartë*)** Çelësi më i vogël që është **≥ k**.

**- Subtree count** dhe **madhësia e BST:** çdo nyjë ruan një fushë count, e cila tregon numrin e nyjëve në subtree të saj. Lejon llogaritjen e madhësisë së BST në kohë konstante dhe implementimin e operacioneve si rank dhe select.

**- Traversimi i BST (***Tree Traversal***)** përdoret për të vizituar të gjitha nyjët. Metodat kryesore:

- **Inorder (***LVR***)**: majtas → nyja → djathtas
- **Preorder (***VLR***)**: nyja → majtas → djathtas
- **Postorder (***LRV***)**: majtas → djathtas → nyja



order of growth of running time of ordered symbol table operations

**- Delete në BST,** bëhet në disa metoda:

- **Lazy approach (***lazy delete***)** ku vendosim vlerën e nyjes në null duke e lënë çelësin në pemë si "gurvarri" (tombstone), çon në mbingarkesë memoriesh dhe përkeqësim të performancës me kalimin e kohës, pasi nyjet e fshira nuk çlirohen kurrë nga memoria. Çmimi ~ 2 ln *N* për insert, search, dhe delete (kur çelësat janë random).
- **DeleteMin** shkohet majtas derisa të gjendet nyjë pa fëmijë të majtë, zëvendësohet me fëmijën e djathtë (nëse ekziston) dhe përditësohen count-et (*1 + size(fëmija_majtas) + size(fëmija_djathtas)*).
- **Hibbard delete,** 0 fëmijë → hiqet nyja, 1 fëmijë → zëvendësohet me fëmijën, 2 fëmijë → zëvendësohet me pasardhësin (minimumi i subtree të djathtë). Nyja fshihet dhe bëhet e disponueshme për garbage collection.

| implementation | guarantee | | | average case | | | ordered ops? | operations on keys |
|---|---|---|---|---|---|---|---|---|
| | search | insert | delete | search hit | insert | delete | | |
| sequential search (linked list) | *N* | *N* | *N* | ½ *N* | *N* | ½ *N* | | `equals()` |
| binary search (ordered array) | lg *N* | *N* | *N* | lg *N* | ½ *N* | ½ *N* | ✓ | `compareTo()` |
| BST | *N* | *N* | *N* | 1.39 lg *N* | 1.39 lg *N* | √*N* | ✓ | `compareTo()` |

other operations also become √N
if deletions allowed

**- Krahasimi me struktura të tjera (***Symbol Tables***)**

- **Linked List**: e ngadaltë, por e thjeshtë
- **Binary Search Array**: kërkim i shpejtë, insert i shtrenjtë
- **BST**: fleksibil, por varet nga balancimi

BST është një strukturë shumë e fuqishme për **symbol tables**, por performanca e saj **varet nga forma e drurit**. Për përdorim real dhe efikas, BST duhet të jetë **i balancuar**. Për këtë arsye përdoren variante të avancuara si **AVL Trees** dhe **Red-Black Trees**.

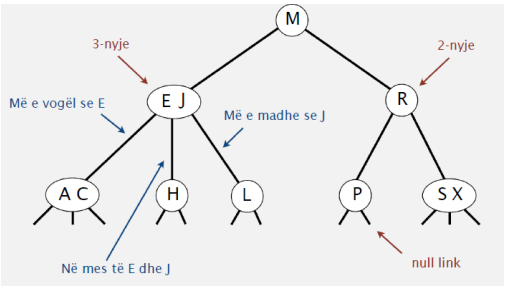
# 11. Balance Search Tree

**2–3 tree** është një **pemë kërkimi e balancuar në mënyrë perfekte**, ku çdo nyje mund të jetë:

- **2-node**: përmban **1 çelës** dhe ka **2 fëmijë**
- **3-node**: përmban **2 çelësa** dhe ka **3 fëmijë**

Pemë 2–3 ruan renditje simetrike, që do të thotë se një inorder traversal i pemës kthen çelësat nga më i vogli tek më i madhi. Një veçori shumë e rëndësishme e 2–3 trees është **balancimi perfekt**:

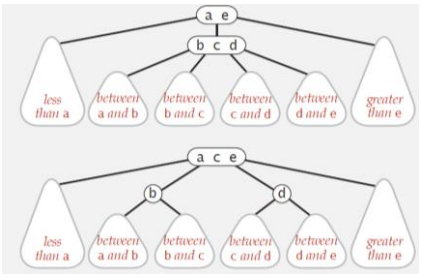
- Çdo rrugë nga **root** deri te një **null link** ka **gjatësi të njëjtë**
- Nuk ekziston rast më i keq më i thellë se rastet e tjera

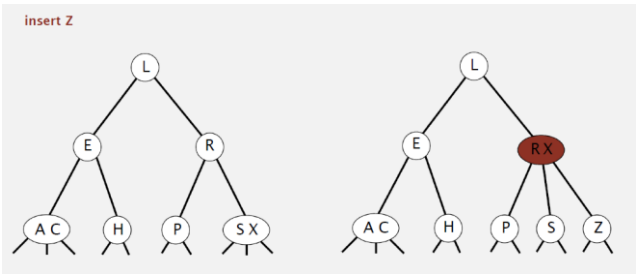


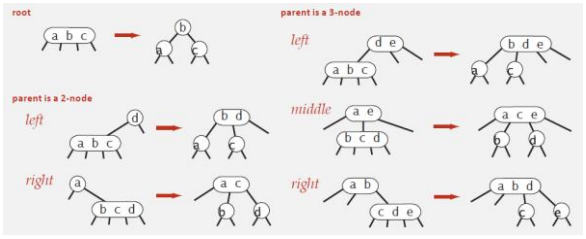

Si rezultat, **lartësia e drurit** është **O(*log N*) all 3 node** në rast të mirë, kurse **O(*lgN*) all 2 node** të keq, dhe search dhe insert gjithmonë ekzekutohen në kohë logaritmike. Gjatë inserimit:

- Nëse futet në një **2-node**, ajo shndërrohet në **3-node**
- Nëse futet në një **3-node**, krijohet përkohësisht një **4-node**
- 4-node ndahet (split) dhe çelësi i mesëm ngrihet te prindi

Këto ndarje janë **transformime lokale** dhe kërkojnë vetëm një numër konstant operacionesh.




| implementation | guarantee | | | average case | | | ordered ops? | key interface |
|---|---|---|---|---|---|---|---|---|
| | search | insert | delete | search hit | insert | delete | | |
| sequential search (unordered list) | *N* | *N* | *N* | ½ *N* | *N* | ½ *N* | | equals() |
| binary search (ordered array) | lg *N* | *N* | *N* | lg *N* | ½ *N* | ½ *N* | ✓ | compareTo() |
| BST | *N* | *N* | *N* | 1.39 lg *N* | 1.39 lg *N* | √*N* | ✓ | compareTo() |
| 2-3 tree | *c* lg *N* | *c* lg *N* | *c* lg *N* | *c* lg *N* | *c* lg *N* | *c* lg *N* | ✓ | compareTo() |

- **Red–Black BST** është një **Binary Search Tree** që implementon logjikën e **2–3 tree** duke përdorur **link-a të ngjyrosur (***të kuq dhe të zi***)**. Qëllimi i tij është të ofrojë:

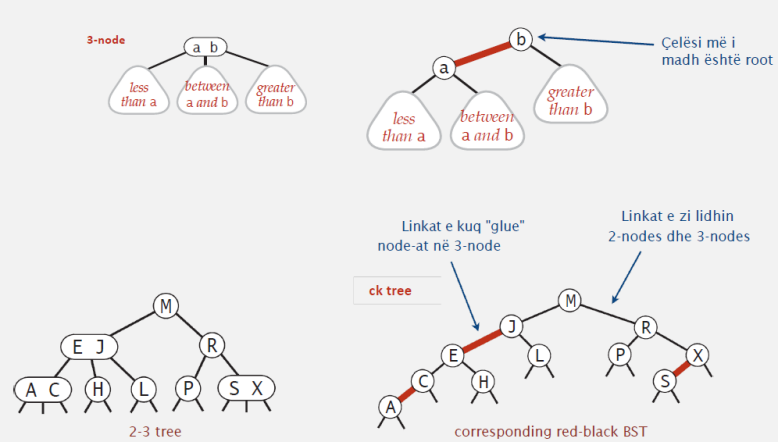
- Performancë **logaritmike të garantuar**
- Implementim më praktik sesa 2–3 trees

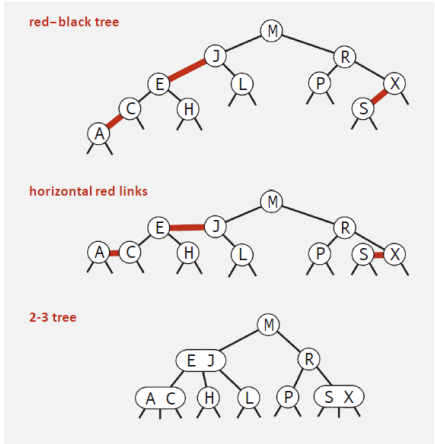
Një red–black tree duhet të plotësojë këto rregulla:

- Asnjë nyje nuk mund të ketë **dy link-a të kuq njëri pas tjetrit**
- Çdo rrugë nga root deri te null link ka **numër të njëjtë link-ash të zi**
- **Link-at e kuq anojnë majtas** (left-leaning)

Këto rregulla sigurojnë atë që quhet perfect black balance.

- Red–black BST është një **pasqyrim 1–me–1** i një 2–3 tree, kjo do të thotë se çdo red–black tree përfaqëson saktësisht një 2–3 tree.

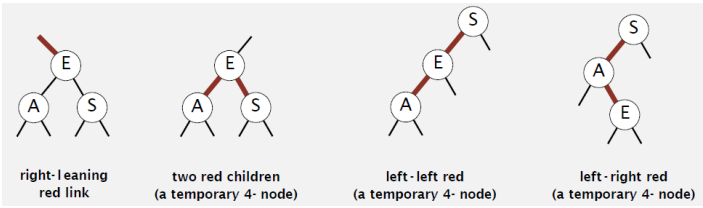


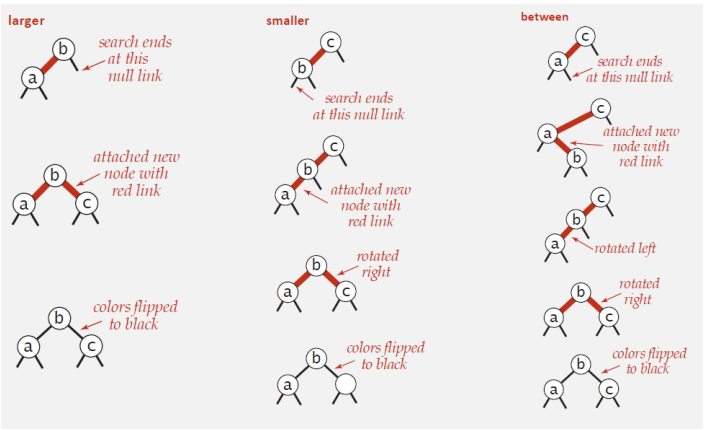



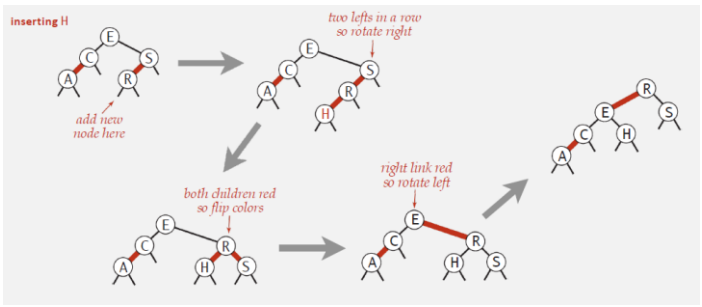

**Performanca e Red–Black BST** falë balancimit lartësia e drurit është **≤ 2·log N** në rastin më të keq. Në praktikë është afërsisht **1.0·log N.** Prandaj:

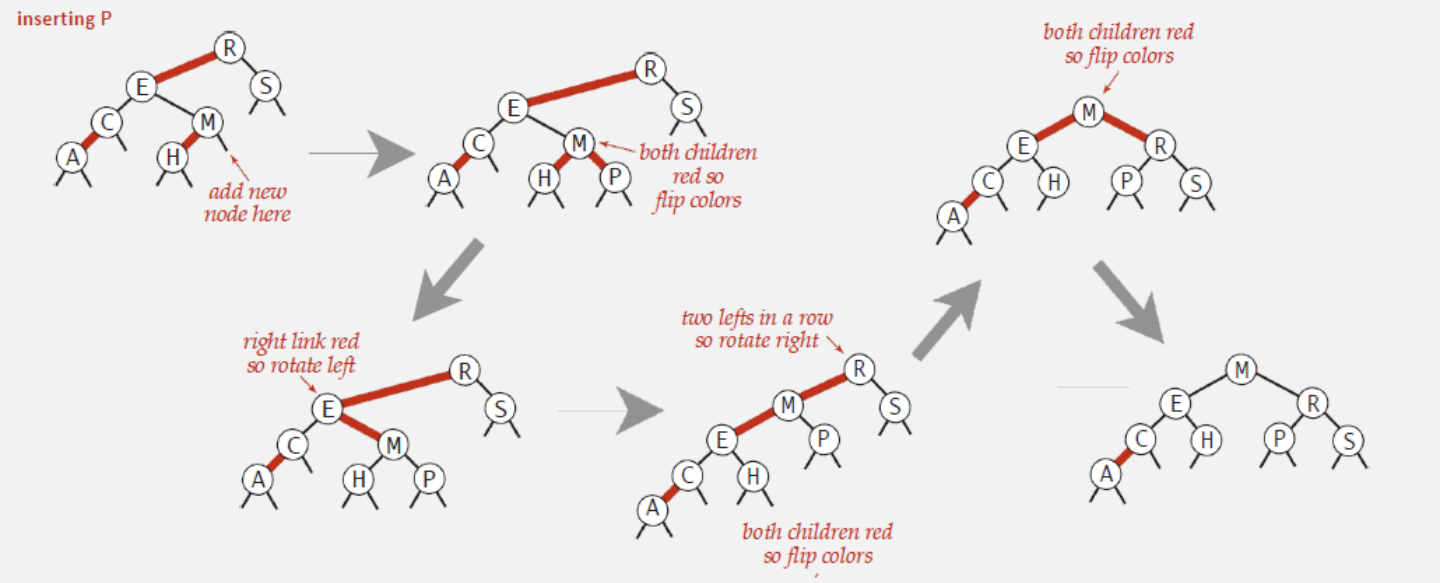
- **search (**njëjtë sit tek BST**), insert dhe delete** ekzekutohen në **O(log N)**
- Performanca është **e garantuar**, jo vetëm mesatare

- Përgjatë operacioneve, të mbahet:
  - Rradhitja simetrike.
  - Balanci perfekt –druri është i zi.






**- Përmbledhje**

- **2–3 tree**: pemë e balancuar perfekt, garanton O(log N)
- **Red–Black BST**: implementim praktik i 2–3 tree me link-a të kuq dhe të zi
- **Avantazhi kryesor**: performancë logaritmike edhe në worst case

| implementation | guarantee | | | average case | | | ordered ops? | key interface |
|---|---|---|---|---|---|---|---|---|
| | search | insert | delete | search hit | insert | delete | | |
| **sequential search (unordered list)** | $N$ | $N$ | $N$ | $\frac{1}{2} N$ | $N$ | $\frac{1}{2} N$ | | `equals()` |
| **binary search (ordered array)** | $\lg N$ | $N$ | $N$ | $\lg N$ | $\frac{1}{2} N$ | $\frac{1}{2} N$ | ✓ | `compareTo()` |
| **BST** | $N$ | $N$ | $N$ | $1.39 \lg N$ | $1.39 \lg N$ | $\sqrt{N}$ | ✓ | `compareTo()` |
| **2-3 tree** | $c \lg N$ | $c \lg N$ | $c \lg N$ | $c \lg N$ | $c \lg N$ | $c \lg N$ | ✓ | `compareTo()` |
| **red-black BST** | $2 \lg N$ | $2 \lg N$ | $2 \lg N$ | $1.0 \lg N$ * | $1.0 \lg N$ * | $1.0 \lg N$ * | ✓ | `compareTo()` |

Lartësia e drurit është **≤2 lg *N*** në rastin më të keq.

Lartësia e drurit është **~ 1.0 lg *N*** në rastin tipik.

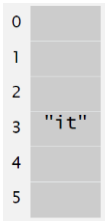
Lartësia e drurit është **~ log*N*** në rastin më të mirë.

Çdo rrugë prej root tek null link-u ka numër të njejtë të linkave të zi. Kurrë nuk ka dy link-a të kuq në seri (të njëpasnjëshëm).

# 12. Hash Tabels

**- Hashing:** Elementet ruhen në një **key-indexed tabelë**, ku indeksi llogaritet nga një **hash funksion** mbi çelësin. Komponentët kryesorë:

- **Hash funksioni** – llogarit indeksin e array-it nga çelësi
- **Equality test** – kontrollon nëse dy çelësa janë të barabartë (equals)
- **Zgjidhja e kolizioneve** – kur dy çelësa mapohen në të njëjtin indeks




Kolizionet janë të pashmangshme pa përdorur memorie jorealiste (kuadratike).

**- Hash code në Java:** Të gjitha klasat në Java trashëgojnë metodën **hashCode()**, e cila kthen një numër int 32-bit. Kërkesa themelore:

- Nëse x.equals(y) → atëherë x.hashCode() == y.hashCode()
- Preferohet që objekte të ndryshme të kenë hashCode të ndryshëm, p.sh nëse !x.equals(y) → atëherë x.hashCode() != y.hashCode()

**Hashing i String-ve:** Java përdor metodën e **Horner-it** për hashing të string-ve. Formula:

$$\mathbf{h} = s[0]\cdot 31^{(L-1)} + \ldots + s[L-3]\cdot 31^2 + s[L-2]\cdot 31^1 + s[L-1]\cdot 31^0$$

```
public final class String
{
   private final char[] s;
   ...

   public int hashCode()
   {
      int hash = 0;
      for (int i = 0; i < length(); i++)
         hash = s[i] + (31 * hash);
      return hash;
   }
}
```

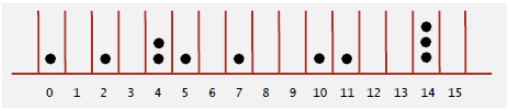
ith character of s

```
String s = "call";
int code = s.hashCode();
```

3045982 = 99·313 + 97·312 + 108·311 + 108·310
= 108 + 31·(108 + 31·(97 + 31·(99)))
(Horner's method)

**- Uniform hashing Supozimi:** çdo çelës ka gjasë të mapohet në cilindo indeks nga 0 deri M−1. Analogia

- Topat dhe koshat: topat hidhen rastësisht në M kosha
- Birthday problem tregon që kolizionet ndodhin shumë shpejt
- Pritet që çdo kosh ka ≥ 1 top pas ~ $M \ln M$ gjuajtjeve.

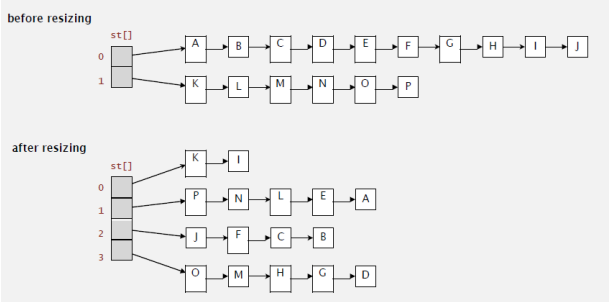



Kolizionet ndodhin kur dy çelësa të ndryshëm prodhojnë të njëjtin vlerë hash, duke përfunduar në të njëjtin indeks në tabelën hash. Sipas Birthday Problem, kolizionet janë të pashmangshme nëse nuk përdorim një sasi të madhe memorie. Sfida kryesore është trajtimi efikas i kolizioneve, siç shihet në shembullin ku hash("times") dhe hash("it") të dyja japin vlerën 3, duke kërkuar zgjidhje si chaining ose linear probing për të ruajtur të dyja çelësat në të njëjtin bucket pa humbur të dhëna.

**- Closed Addressing (***Separate Chaining***)** i njohur edhe si **separate chaining**, është një metodë për zgjidhjen e kolizioneve ku çdo indeks i array-t përmban një strukturë tjetër të të dhënave, zakonisht një listë të lidhur. Kur dy ose më shumë çelësa hashohen në të njëjtin indeks:

- ata ruhen në të njëjtin indeks, por
- lidhen në një zinxhir (chain)

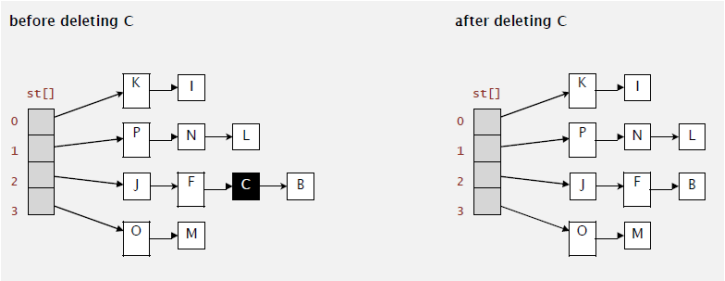
Në këtë rast, array-ja shërben vetëm si pikë hyrëse, ndërsa elementet realë ruhen në strukturat e varura nga ai indeks. **Karakteristikat:**




- Insert/search bëhen vetëm në zinxhirin përkatës
- Performanca varet nga raporti N/M = sa çelësa ka mesatarisht në çdo bucket. Nëse kemi 100 çelësa dhe 20 bucket-e → 100/20 = 5 çelësa mesatarisht për bucket.

**Resizing**

- Dyfishohet M kur N/M ≥ 8
- Përgjysmohet M kur N/M ≤ 2




Për tu bërë delete një çelës (dhe vlera përcjellëse) vetëm **largohet zingjiri** që lidh çelësin në fjalë.

**- Open addressing** është një metodë për zgjidhjen e kolizioneve në hashing, ku të gjitha çiftet çelës–vlerë ruhen drejtpërdrejt në array-n kryesor. Kur ndodh një kolizion (dy çelësa hashohen në të njëjtin indeks), elementi i ri vendoset në një pozicion tjetër të lirë brenda të njëjtit array, duke përdorur një strategji kërkimi si:

- **linear probing:** $h(x)=(f(x)+i)\ modM$
- **quadratic probing:** $h(x)=(f(x)+i+i^2)\ modM$
- **double hashing** $h_1(x)=f(x)\ modM,\ \ h_2(x)=M-(f(x)\ modM),\ h(x)=(\ h_1(x\ )+i+h_2(x))modM$

Kur ndodh kolizion provohet indeksi i ardhshëm (i+1, i+2, …). Këtu M duhet të jetë më i madh se N dhe ka problem **clustering** (grumbullim). **Resizing:**

- Dyfishohet M kur N/M ≥ 1/2
- Përgjysmohet M kur N/M ≤ 1/8

```
x.hashCode() does not change
but hash(x) can change
```

Clustering është formimi i blloqeve të njëpasnjëshme të bucket-eve të zëné në hash tables, që ndodh kur çelësat e rinj kanë tendencë të hashohen në mes të këtyre blloqeve duke i zgjatur ato dhe duke ngadalësuar operacionet, sepse çdo insertim i ri duhet të "probojë" përmes gjithë cluster-it për të gjetur një vend bosh. Zakonisht ndodh ne Linear Probing sepse i gjethmon rritet vetëm për një (length-i mesatar $N/M\leq½$.).

- **Separate Chaining** performancë degradohet gradualisht, më rezistente ndaj hash funksioneve të dobëta.

- **Linear Probing** më pak hapësirë e humbur, performancë më e mirë cache.

| implementation | guarantee | | | average case | | | ordered ops? | key interface |
|---|---|---|---|---|---|---|---|---|
| | search | insert | delete | search hit | insert | delete | | |
| sequential search (unordered list) | $N$ | $N$ | $N$ | $\frac{1}{2} N$ | $N$ | $\frac{1}{2} N$ | | equals() |
| binary search (ordered array) | $\lg N$ | $N$ | $N$ | $\lg N$ | $\frac{1}{2} N$ | $\frac{1}{2} N$ | ✓ | compareTo() |
| BST | $N$ | $N$ | $N$ | $1.39 \lg N$ | $1.39 \lg N$ | $\sqrt{N}$ | ✓ | compareTo() |
| red-black BST | $2 \lg N$ | $2 \lg N$ | $2 \lg N$ | $1.0 \lg N$ | $1.0 \lg N$ | $1.0 \lg N$ | ✓ | compareTo() |
| separate chaining | $N$ | $N$ | $N$ | 3-5 * | 3-5 * | 3-5 * | | equals() hashCode() |
| linear probing | $N$ | $N$ | $N$ | 3-5 * | 3-5 * | 3-5 * | | equals() hashCode() |

- **Sulmet me Kompleksitet Algoritmik** që përdorin të metat e algoritmeve për të bërë sistemet shumë të ngadalta, pa nevojë për shumë burime. **Problemi kryesor:** Sulmet **DoS (Denial of Service)** ku sulmuesit targetojnë hash tables duke gjetur çelësa që të gjithë hash-ohen në të njëjtin bucket, duke kthyer **O(1)** në **O(n).** Hash-ohen të gjithë në **të njëjtin indeks** (kolizione maksimale)
  - Ngadalësojnë operacionet nga **O(1)** në **O(N)**
  - Shkatërrojnë performancën e tabelave hash

- **Zgjidhje** të përshtatshme:
  - **One-way hash functions** (SHA-256, MD5) – kriptografikisht të sigurt, karakteristikat kyçe:
    - **Njëkahëshe (One-way):** Nga hash-i nuk mund të rikthehet çelësi
    - **Rezistencë ndaj kolizioneve:** Vështirë të gjesh dy inpute me të njëjtin output
    - **Avalanche effect:** Një ndryshim i vogël në input prodhon hash krejtësisht të ndryshëm
  - **Sipas nevojës:** Për tabela simbolike, mjaftojnë hash funksione të thjeshta por të dizajnuara mirë
  - **Java:** HashMap përdor hashCode() që mund të overload-ohet

- **Two-Probe Hashing (**variant i separate chaining**),** hasho çelësin në **dy pozicione të ndryshme**, vendose në **listën më të shkurtër,** p.sh hash1("A")=3, hash2("A")=7 → vendose në bucket 3 ose 7, cilido ka listë më të shkurtër**.** Redukton gjatësinë e listës më të gjatë në **logN.**

- **Double Hashing (**variant i linear probing**),** përdor **dy funksione hash** - një për pozicionin fillestar, një për "hapen" e kërkimit**:** (hash1(key) + i × hash2(key)) % M**.** Eliminon clustering, lejon tabelën të mbushet 99%

- **Cuckoo Hashing (**linear probing variant**),** çelësi ka **dy vende të mundshme**; nëse i zënë të dyja, "përzë" çelësin ekzistues. Mund të ketë cikle të pafundme (nevojitet rehash) por **O(1) worst-case** për search (jo vetëm average!)

| Hash Tables | Balanced Search Trees |
|---|---|
| **O(1)** mesatar për search/insert | **O(log N)** garantuar |
| **Nuk mbështesin renditjen** | **Mbështesin renditjen** (in-order) |
| Varësia nga hash funksioni | Strukturë e balancuar (AVL, Red-Black) |
| Më pak memorie (pa pointers shtesë) | Më shumë memorie (pointers) |
| **Java:** HashMap, HashSet, IdentityHashMap | **Java:** TreeMap, TreeSet |

**-** Kur të përdorim çfarë:

- **Hash Table:** Kur duhet shpejtësi maksimale dhe nuk na duhen çelësat e renditur.
- **Balanced Tree:** Kur duhen operacione të renditura ose performancë e garantuar.